A PROJECT REPORT ON

Human Resources and Employee Analytics

Workforce analysis using Python (Pandas, NumPy and Matplotlib)

Part 1: Employee Performance and Productivity Analysis

Part 2: Employee Attrition and Retention Analysis

Part 3: Recruitment and Hiring Funnel Analysis

Part 4: Salary Distribution and Compensation Analysis

Part 5: Employee Training and Skill Development Analysis

About this report

This report explains, step by step, how five data files were cleaned and analysed using Python (Pandas, NumPy and Matplotlib) in Jupyter Notebooks: an employee performance file (Part 1), an employee attrition file (Part 2), a recruitment file (Part 3), a salary file (Part 4) and an employee training file (Part 5). The aim is to understand employee performance and productivity (Part 1), why employees leave the company (Part 2), how candidates move through the hiring process (Part 3), how salaries are distributed (Part 4) and how training improves employee scores and how much it costs (Part 5). Every step has a short explanation in simple language and a screenshot of the code and its result.

Part 1 dataset: Task1.csv

Columns (9): Employee ID, Department, Designation, Joining Date, Performance Score, Projects Completed, Working Hours, Experience Years, Salary.

Part 2 dataset: Task 2.csv (2,678 rows, 9 columns: Employee ID, Department, Age, Gender, Job Satisfaction, Salary, Years at Company, Promotion History, Attrition Status). Its summary is at the start of Part 2.

Part 3 dataset: Task 3.csv (2,930 rows, 10 columns: Candidate ID, Job Role, Application Date, Screening Status, Interview Score, Interview Status, Offer Status, Joining Status, Recruitment Source, Joining Date). Its summary is at the start of Part 3.

Part 4 dataset: Task 4.csv (2,750 rows, 8 columns: Employee ID, Department, Designation, Experience, Education Level, Salary, Performance Rating, Location). Its summary is at the start of Part 4.

Part 5 dataset: Task 5.csv (2,750 rows, 8 columns: Employee ID, Department, Training Program, Training Hours, Pre-Training Score, Post-Training Score, Training Cost, Productivity Score). Its summary is at the start of Part 5.

Part 1 columns in the dataset

Column	Meaning

Column	Meaning
Employee ID	Unique identification of each employee
Department	Department in which the employee works
Designation	Job position of the employee
Joining Date	Date on which the employee joined
Performance Score	Performance score of the employee
Projects Completed	Number of projects completed
Working Hours	Hours spent working
Experience Years	Total work experience in years
Salary	Salary received by the employee

Quick summary of results (Part 1)

Item	Result
Records in raw file	2,755 rows x 9 columns (24,795 values)
Missing values found	78 (Joining Date 10, Performance Score 17, Projects Completed 15, Working Hours 20, Salary 16)
Department names	12 different spellings cleaned into 7 departments
Invalid values fixed	Performance Score outside 0-100, negative Projects Completed, Working Hours above 80, negative Salary
Rows removed: exact duplicates	5
Records after cleaning	2,750 rows x 9 columns
Average performance score	74.10 (middle value 74.10, standard deviation 10.08)
Highest performing department	IT (average score 76.88)
Lowest performing department	Customer Support (average score 70.56)

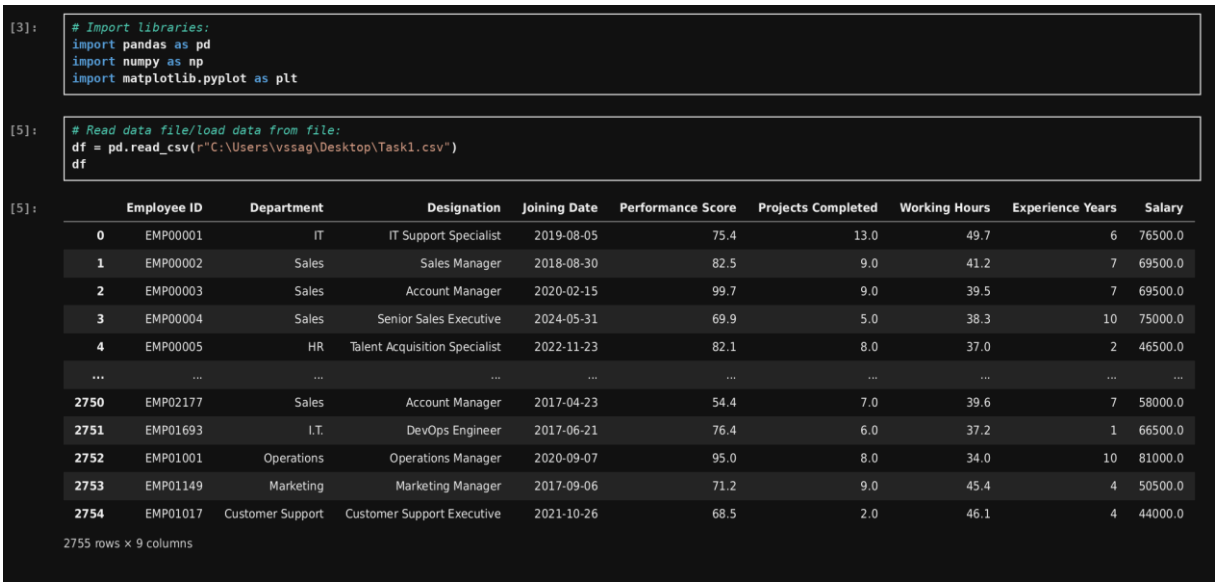
PART 1: Employee Performance and Productivity Analysis

Part 1 - Task 1: Load the Dataset and Examine Its Structure

First, the libraries were imported and the CSV file was loaded into a Pandas DataFrame called `df`. Then the data was looked at in several ways to understand its size, column names, data types and a few sample rows. This is the first step in every analysis, because it shows what kind of problems the data may have.

1.1 Import libraries and load the data

Pandas is used for tables and calculations, NumPy for numbers, and Matplotlib for charts. The last line of the second cell, df, shows a preview of the data: the first and last five rows and the size of the table (2,755 rows x 9 columns).



[3]:
```
# Import libraries:
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

[5]:
```
# Read data file/load data from file:
df = pd.read_csv(r"C:\Users\vssag\Desktop\Task1.csv")
df
```

[5]:

	Employee ID	Department	Designation	Joining Date	Performance Score	Projects Completed	Working Hours	Experience Years	Salary
0	EMP00001	IT	IT Support Specialist	2019-08-05	75.4	13.0	49.7	6	76500.0
1	EMP00002	Sales	Sales Manager	2018-08-30	82.5	9.0	41.2	7	69500.0
2	EMP00003	Sales	Account Manager	2020-02-15	99.7	9.0	39.5	7	69500.0
3	EMP00004	Sales	Senior Sales Executive	2024-05-31	69.9	5.0	38.3	10	75000.0
4	EMP00005	HR	Talent Acquisition Specialist	2022-11-23	82.1	8.0	37.0	2	46500.0
...	...	...	...	...	...	...	...	...	...
2750	EMP02177	Sales	Account Manager	2017-04-23	54.4	7.0	39.6	7	58000.0
2751	EMP01693	I.T.	DevOps Engineer	2017-06-21	76.4	6.0	37.2	1	66500.0
2752	EMP01001	Operations	Operations Manager	2020-09-07	95.0	8.0	34.0	10	81000.0
2753	EMP01149	Marketing	Marketing Manager	2017-09-06	71.2	9.0	45.4	4	50500.0
2754	EMP01017	Customer Support	Customer Support Executive	2021-10-26	68.5	2.0	46.1	4	44000.0

2755 rows × 9 columns

Screenshot: importing libraries and loading the CSV file

1.2 First look at the data

`sample(5)` shows 5 random rows, `head()` shows the first five rows and `tail()` shows the last five rows. One problem is already visible: the last rows show department names written differently, such as "I.T." instead of "IT".

```
[3]: df.sample(5) # It will give random 5 rows.
```

[3]:

	Employee ID	Department	Designation	Joining Date	Performance Score	Projects Completed	Working Hours	Experience Years	Salary
348	EMP00349	Finance	Finance Manager	2019-05-23	65.2	6.0	46.8	7	64500.0
1683	EMP01684	Sales	Senior Sales Executive	2017-09-14	75.4	7.0	42.6	6	62500.0
2678	EMP02679	HR	HR Executive	2023-10-05	88.4	8.0	40.7	4	54000.0
1315	EMP01316	Finance	Payroll Specialist	2023-01-11	75.6	10.0	47.5	9	71000.0
495	EMP00496	Operations	Operations Manager	2016-04-30	80.5	9.0	34.7	7	69500.0

```
[4]: df.head() # It shows top five rows
```

[4]:

	Employee ID	Department	Designation	Joining Date	Performance Score	Projects Completed	Working Hours	Experience Years	Salary
0	EMP00001	IT	IT Support Specialist	2019-08-05	75.4	13.0	49.7	6	76500.0
1	EMP00002	Sales	Sales Manager	2018-08-30	82.5	9.0	41.2	7	69500.0
2	EMP00003	Sales	Account Manager	2020-02-15	99.7	9.0	39.5	7	69500.0
3	EMP00004	Sales	Senior Sales Executive	2024-05-31	69.9	5.0	38.3	10	75000.0
4	EMP00005	HR	Talent Acquisition Specialist	2022-11-23	82.1	8.0	37.0	2	46500.0

```
[5]: df.tail() # It shows last five rows
```

[5]:

	Employee ID	Department	Designation	Joining Date	Performance Score	Projects Completed	Working Hours	Experience Years	Salary
2750	EMP02177	Sales	Account Manager	2017-04-23	54.4	7.0	39.6	7	58000.0
2751	EMP01693	I.T.	DevOps Engineer	2017-06-21	76.4	6.0	37.2	1	66500.0
2752	EMP01001	Operations	Operations Manager	2020-09-07	95.0	8.0	34.0	10	81000.0
2753	EMP01149	Marketing	Marketing Manager	2017-09-06	71.2	9.0	45.4	4	50500.0
2754	EMP01017	Customer Support	Customer Support Executive	2021-10-26	68.5	2.0	46.1	4	44000.0

Screenshot: df.sample(5), df.head() and df.tail()

1.3 Shape, dimensions and column names

The dataset has **2,755 rows and 9 columns**, so 24,795 values in total. `df.ndim` is 2, which means the data is a two-dimensional table.

```
[6]: df.shape # It will give dataset size(rows,column)

[6]: (2755, 9)

[17]: df.ndim # It will give the dimensions.

[17]: 2

[7]: print(df.columns.tolist()) # It shows All column names in a list.

['Employee ID', 'Department', 'Designation', 'Joining Date', 'Performance Score', 'Projects Completed',
 'Working Hours', 'Experience Years', 'Salary']
```

Screenshot: shape, dimensions and column names

1.4 Dataset information and data types

The `info()` output shows how many values are filled in each column and its data type. Joining Date is stored as plain text (object) instead of a date, so it must be converted later. Performance Score, Projects Completed, Working Hours and Salary are decimal numbers (float64), and Experience Years is a whole number (int64).

```
[8]:  df.info() # It will give Dataset structure.

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2755 entries, 0 to 2754
Data columns (total 9 columns):
 #   Column              Non-Null Count  Dtype
---  ------              --------------  -----
 0   Employee ID         2755 non-null   object
 1   Department          2755 non-null   object
 2   Designation         2755 non-null   object
 3   Joining Date        2745 non-null   object
 4   Performance Score   2738 non-null   float64
 5   Projects Completed  2740 non-null   float64
 6   Working Hours       2735 non-null   float64
 7   Experience Years    2755 non-null   int64
 8   Salary              2739 non-null   float64
dtypes: float64(4), int64(1), object(4)
memory usage: 193.8+ KB

[10]: df.dtypes # it will shows Data types of columns

[10]: Employee ID            object
      Department             object
      Designation            object
      Joining Date           object
      Performance Score     float64
      Projects Completed    float64
      Working Hours         float64
      Experience Years        int64
      Salary                float64
      dtype: object
```

Screenshot: df.info() and df.dtypes

1.5 Statistical summary

The `describe()` table helps to spot values that are not possible in real life:

- **Performance Score:** minimum is -5 and maximum is 108, but a score should be between 0 and 100.
- **Projects Completed:** minimum is -2, and a number of projects cannot be negative.
- **Working Hours:** maximum is 95 hours, while the middle value (median) is only 42.2 and 75% of employees work 45.8 hours or less.

```
[9]:  df.describe() # It shows statistical summary of the dataset
```

[9]:

	Performance Score	Projects Completed	Working Hours	Experience Years	Salary
count	2738.000000	2740.000000	2735.000000	2755.000000	2739.000000
mean	74.079511	7.324453	42.131115	5.090381	60487.586710
std	10.247793	3.046587	5.587637	3.046510	13562.479906
min	-5.000000	-2.000000	25.000000	0.000000	-12000.000000
25%	67.200000	5.000000	38.300000	3.000000	51000.000000
50%	74.200000	7.000000	42.200000	5.000000	59000.000000
75%	80.900000	9.000000	45.800000	7.000000	69500.000000
max	108.000000	22.000000	95.000000	16.000000	106500.000000

Screenshot: df.describe()

1.6 Missing values and unique values

In total **78** values are missing: Working Hours (20), Performance Score (17), Salary (16), Projects Completed (15) and Joining Date (10). This is only about 0.3% of all values, so the data can be fixed instead of deleted. `nunique()` shows that Employee ID has 2,750 unique values although there are 2,755 rows, which means 5 rows are repeated. It also shows 12 different Department names, even though a company like this has only 7 departments, which points to inconsistent spelling.

```
[11]:  df.isnull().any() # Check any missing value in dataset by each column.

[11]:  Employee ID           False
       Department            False
       Designation           False
       Joining Date           True
       Performance Score      True
       Projects Completed     True
       Working Hours          True
       Experience Years      False
       Salary                 True
       dtype: bool

[12]:  df.isnull().sum() # It shows all misiing not/nll in each column.

[12]:  Employee ID            0
       Department             0
       Designation            0
       Joining Date          10
       Performance Score     17
       Projects Completed    15
       Working Hours         20
       Experience Years       0
       Salary                16
       dtype: int64

[13]:  df.nunique() # It will give The count of nunique values.

[13]:  Employee ID           2750
       Department              12
       Designation             27
       Joining Date          1938
       Performance Score      476
       Projects Completed      23
       Working Hours          289
       Experience Years        17
       Salary                 146
       dtype: int64
```

Screenshot: missing values and unique values per column

Part 1 - Task 2: Identify and Handle Missing Values, Duplicate Records and Inconsistent Entries

Before fixing anything, one rule was followed: never fill a missing or wrong value in a way that creates false information. Different columns hold different kinds of data, so each column was handled with its own method. Two methods were used:

- **Fill with the mean (average):** used for Performance Score, because it is a score on a fixed 0 to 100 scale and the average represents a typical employee.
- **Fill with the median:** used for Projects Completed, Working Hours and Salary. The median is the middle value, so a few extreme values (like 95 working hours) do not pull it up or down.

Invalid values (such as a score above 100) were first changed to NaN (missing) and then filled using the method of that column.

2.1 Summary: how every column was handled

Column	Problem found	How it was handled	Why
Department	12 different spellings (H.R., Human Resources, I.T., Cust. Support, hr, customer support), extra spaces, mixed case	Trimmed spaces, converted to upper case, replaced with standard names; now 7 departments	Same department must be counted as one group
Designation	27 unique designations checked	No change needed	Names were already consistent
Performance Score	17 missing; values below 0 or above 100 (min -5, max 108)	Invalid values set to NaN, all missing filled with the mean (74.10)	A score must be between 0 and 100
Projects Completed	15 missing; negative values (min -2)	Invalid values set to NaN, filled with the median (7)	A project count cannot be negative
Working Hours	20 missing; one value of 95 hours	Values above 80 set to NaN, filled with the median (42.2)	95 hours is not realistic and would distort the average
Salary	16 missing; negative values	Negative values set to NaN, filled with the median	Salary cannot be negative
Joining Date	10 missing; stored as text	Converted to datetime (Task 3)	Dates are needed for date-based analysis
All rows	5 exact duplicate rows	Removed with drop_duplicates()	Same employee must be counted once

2.2 Department: inconsistent entries

The unique department names were printed first. The same department appeared in many forms, for example "HR", "hr" and "Human Resources", or "IT" and "I.T.". This would split one department into several groups. The names were cleaned in two steps: first all text was trimmed and converted to upper case, then the different spellings were replaced with one standard name. After cleaning, only 7 departments remain.

```
[19]: print(df["Department"].unique().tolist()) # Check unique department names in a list

['IT', 'Sales', 'HR', 'Operations', 'Finance', 'Customer Support', 'Marketing', 'Human Resources', 'Cust.
  Support', 'I.T.', 'hr', 'customer support']

[20]: df["Department"] = df["Department"].str.strip().str.upper()

      df["Department"] = df["Department"].replace({
          "H.R.": "HR",
          "HUMAN RESOURCES": "HR",      # Handle Departments consistency.
          "I.T.": "IT",
          "CUST. SUPPORT": "CUSTOMER SUPPORT"
      })
      print(df["Department"].unique()) # Recheck

      ['IT' 'SALES' 'HR' 'OPERATIONS' 'FINANCE' 'CUSTOMER SUPPORT' 'MARKETING']
```

Screenshot: department names before and after cleaning

2.3 Designation

The designation names were checked in the same way. There are 27 designations and all of them are written consistently, so no change was needed.

```
[21]: print(df["Designation"].unique().tolist()) # Check unique Designation names in a list.

['IT Support Specialist', 'Sales Manager', 'Account Manager', 'Senior Sales Executive', 'Talent
  Acquisition Specialist', 'Operations Executive', 'Accountant', 'Customer Support Executive', 'Data
  Analyst', 'HR Executive', 'DevOps Engineer', 'Payroll Specialist', 'QA Engineer', 'Logistics
  Coordinator', 'SEO Analyst', 'HR Manager', 'Team Lead - Support', 'Operations Manager', 'Process
  Improvement Analyst', 'Sales Executive', 'Financial Analyst', 'Marketing Executive', 'Content
  Strategist', 'Marketing Manager', 'Finance Manager', 'Customer Success Specialist', 'Software
  Engineer']
```

Screenshot: unique designation names

2.4 Performance Score: invalid and missing values

The summary showed a minimum of -5 and a maximum of 108. Scores below 0 or above 100 were set to NaN. After this, the minimum is 34.9 and the maximum is 100, which is a valid range. Then all missing scores were filled with the mean.

```
[23]: df["Performance Score"].describe() # It shows Statistical summary.

[23]: count    2738.000000
      mean       74.079511
      std        10.247793
      min        -5.000000
      25%        67.200000
      50%        74.200000
      75%        80.900000
      max       108.000000
      Name: Performance Score, dtype: float64

[24]: df.loc[(df["Performance Score"] < 0) | (df["Performance Score"] > 100),
          "Performance Score"] = None # Handle invalid scores

[25]: df["Performance Score"].describe()# It shows Statistical summary.

[25]: count    2736.000000
      mean       74.096016
      std        10.118626
      min        34.900000
      25%        67.200000
      50%        74.200000
      75%        80.900000
      max       100.000000
      Name: Performance Score, dtype: float64

[26]: df["Performance Score"] = df["Performance Score"].fillna(df["Performance Score"].mean()) # Replace with average.
```

Screenshot: Performance Score before and after handling

2.5 Projects Completed: invalid and missing values

The summary showed a minimum of -2, which is not possible. Negative values were set to NaN and then all missing values were filled with the median, which is 7. After this, the count is 2,755 and the minimum is 0.

```
[27]: df["Projects Completed"].describe() # It shows Statistical summary.

[27]: count    2740.000000
      mean        7.324453
      std         3.046587
      min        -2.000000
      25%         5.000000
      50%         7.000000
      75%         9.000000
      max        22.000000
      Name: Projects Completed, dtype: float64

[28]: df.loc[(df["Projects Completed"] <0) | (df["Projects Completed"]>100),
          "Projects Completed"] = None     # Handle invalid scores

[29]: df["Projects Completed"] = df["Projects Completed"].fillna(df["Projects Completed"].median())# Replace with average.

[30]: df["Projects Completed"].describe() # It shows Statistical summary.

[30]: count    2755.000000
      mean        7.325953
      std         3.033180
      min         0.000000
      25%         5.000000
      50%         7.000000
      75%         9.000000
      max        22.000000
      Name: Projects Completed, dtype: float64
```

Screenshot: Projects Completed before and after handling

2.6 Working Hours: unusually high values

The most common working hours were checked, and values above 80 hours were listed. Only one value was found: 95 hours (row 1155). It was treated as unrealistic, set to NaN and filled with the median. After this, the maximum working hours is 60.8.

```
[31]: df["Working Hours"].describe() # It shows Statistical summary.

[31]: count    2735.000000
      mean       42.131115
      std         5.587637
      min        25.000000
      25%        38.300000
      50%        42.200000
      75%        45.800000
      max        95.000000
      Name: Working Hours, dtype: float64

[33]: df["Working Hours"].value_counts().head(10)  # # Check the most common working hours

[33]: Working Hours
      42.2    28
      42.9    27
      43.0    26
      44.5    24
      44.4    24
      42.5    23
      41.5    23
      38.7    23
      41.6    23
      44.8    23
      Name: count, dtype: int64

[34]: print(df[df["Working Hours"] > 80]["Working Hours"]) ## Check unusually high working hours

      1155    95.0
      Name: Working Hours, dtype: float64

[35]: df.loc[df["Working Hours"] > 80, "Working Hours"] = None  # Handle unusually high working hours

[36]: df["Working Hours"].describe()

[36]: count    2734.000000
      mean       42.111778
      std         5.496363
      min        25.000000
      25%        38.300000
      50%        42.200000
      75%        45.800000
      max        60.800000
      Name: Working Hours, dtype: float64

[37]: df["Working Hours"] = df["Working Hours"].fillna(df["Working Hours"].median()) # Replace with median
```

Screenshot: Working Hours: checking and handling values above 80

2.7 Salary: negative values

A salary cannot be negative, so negative values were set to NaN and filled with the median. After this, the minimum salary is 23,000.

```
[38]: df.loc[df["Salary"] < 0, "Salary"] = None # Handle negative salary values

[39]: df["Salary"].describe()

[39]: count      2738.000000
      mean      60514.061359
      std       13493.983133
      min       23000.000000
      25%       51000.000000
      50%       59000.000000
      75%       69500.000000
      max      106500.000000
      Name: Salary, dtype: float64

[40]: df["Salary"] = df["Salary"].fillna(df["Salary"].median()) #Replace missing salary values with the median
```

Screenshot: Salary: handling negative values

2.8 Duplicate records

`duplicated().sum()` showed **5** exact duplicate rows. A duplicate is a row repeated exactly, which would count the same employee twice. These rows were removed with `drop_duplicates()`, and a second check confirmed that 0 duplicates remain. The dataset now has 2,750 rows, equal to the number of unique Employee IDs.

```
[42]: df.duplicated().sum() # Check duplicate rows.

[42]: np.int64(5)

[43]: df =  df.drop_duplicates() # Remove duplicated rows.

[44]: df.duplicated().sum() # Check again for confirmation

[44]: np.int64(0)
```

Screenshot: duplicate rows before and after removal

2.9 Final check

After cleaning, `isna().sum()` was run again. Performance Score, Projects Completed, Working Hours and Salary now show 0 missing values. Joining Date still has 10 missing values; these were left unchanged because Joining Date is not used in any calculation of this analysis.

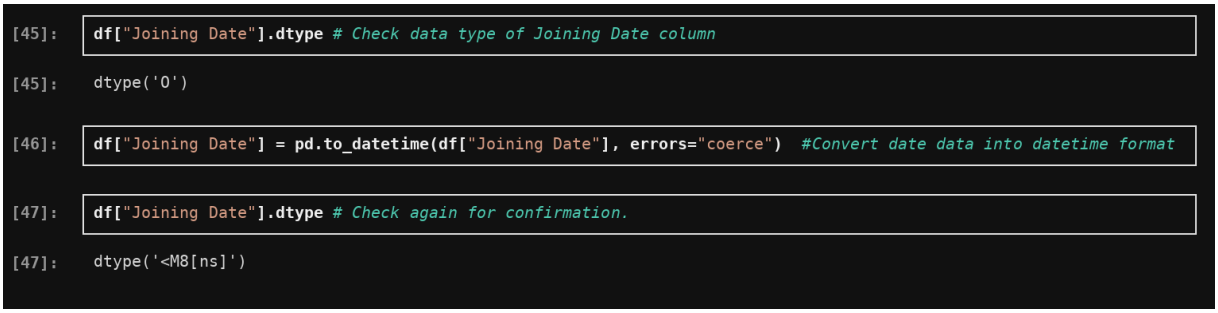
```
[41]: df.isna().sum() # Recheck.

[41]: Employee ID            0
      Department             0
      Designation            0
      Joining Date          10
      Performance Score      0
      Projects Completed     0
      Working Hours          0
      Experience Years       0
      Salary                 0
      dtype: int64
```

Screenshot: missing values after cleaning

Part 1 - Task 3: Convert Joining Date into DateTime Format

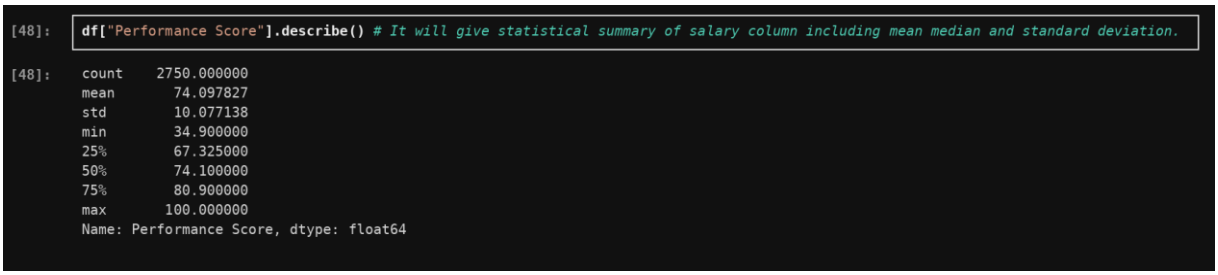
Joining Date was stored as plain text (dtype `O`, which means object). Text dates cannot be sorted or used for date calculations, so the column was converted to a real date-time type with `pd.to_datetime(..., errors="coerce")`. The option `errors="coerce"` means that any value that is not a valid date becomes NaT (Not a Time) instead of causing an error. After conversion the type is `datetime64[ns]`.



Screenshot: Joining Date data type before and after converting to datetime

Part 1 - Task 4: Mean, Median, Minimum, Maximum and Standard Deviation of Performance Scores

These values show the average performance and how much the scores differ between employees. The mean is the average, the median is the middle value, the minimum and maximum are the lowest and highest scores, and the standard deviation shows how spread out the scores are around the mean. All of them come from one command, `describe()`.



Screenshot: statistical summary of Performance Score

Statistic	Performance Score
Count	2,750
Mean	74.10
Median	74.10
Minimum	34.90
Maximum	100.00
Standard deviation	10.08

The mean and the median are almost the same (74.10), which means the scores are balanced and not pulled by extreme values. The standard deviation of about 10 means that most employees score within roughly 10 points above or below the average.

Part 1 - Task 5: Department-wise Average Performance and Project Completion

Using `groupby("Department")`, the data was split into one group per department and the average was calculated for each group. This allows a direct comparison of performance and productivity between departments.

5.1 Average projects completed by department

Project completion is very similar in all departments, between 7.13 and 7.57 projects on average. Marketing is highest (7.57) and Finance is lowest (7.13).

```
[49]: department_avg =  df.groupby("Department") ["Projects Completed"].mean() # Calculate Department wise aveargе Performance
      print(department_avg)

      Department
      CUSTOMER SUPPORT    7.296875
      FINANCE             7.130719
      HR                  7.190045
      IT                  7.388316
      MARKETING           7.571429
      OPERATIONS          7.263158
      SALES               7.396378
      Name: Projects Completed, dtype: float64
```

Screenshot: average projects completed in each department

5.2 Average performance score by department

Performance scores differ more between departments than project counts do. IT is highest (76.88) and Customer Support is lowest (70.56).

```
[50]: avg_completion = df.groupby("Department")["Performance Score"].mean() # It shows avrage project completion rates.
      avg_completion

[50]: Department
      CUSTOMER SUPPORT    70.558802
      FINANCE             74.849987
      HR                  71.925339
      IT                  76.882447
      MARKETING           73.809369
      OPERATIONS          74.993312
      SALES               73.338584
      Name: Performance Score, dtype: float64
```

Screenshot: average performance score in each department

5.3 Combined result

Department	Average Performance Score	Average Projects Completed
IT	76.88	7.39
Operations	74.99	7.26
Finance	74.85	7.13
Marketing	73.81	7.57
Sales	73.34	7.40
HR	71.93	7.19
Customer Support	70.56	7.30

Part 1 - Visualization Tasks

Three charts were created to make the results easier to see: a bar chart, a scatter plot and a box plot.

Task 6: Bar Chart - Department-wise Average Performance

The bar chart compares the average performance score of every department. The value is printed above each bar, so the highest and lowest departments can be seen at once.

[6]:

```
department_avg = df.groupby("Department")["Performance Score"].mean()

plt.figure(figsize=(12, 7), facecolor="#F5F3F8")
ax = plt.gca()
ax.set_facecolor("#FFFFFF")

departments = department_avg.index
performance = department_avg.values

colors = ["#594B8A", "#7665A8", "#9384BC", "#A99BCB",
          "#7B6D9E", "#B5A9CF", "#C5BBDD"]

bars = plt.bar(
    departments, performance,
    width=0.6,
    color=colors[:len(departments)],
    edgecolor="#E5E0ED",
    linewidth=1.2
)

plt.ylim(0, max(performance) + 8)

plt.bar_label(bars, fmt="%.2f", padding=4,
              fontsize=9, fontweight="bold", color="#403653")

plt.title("Average Performance Score Across Departments",
          color="#403653", fontsize=14,
          fontweight="bold", pad=15)

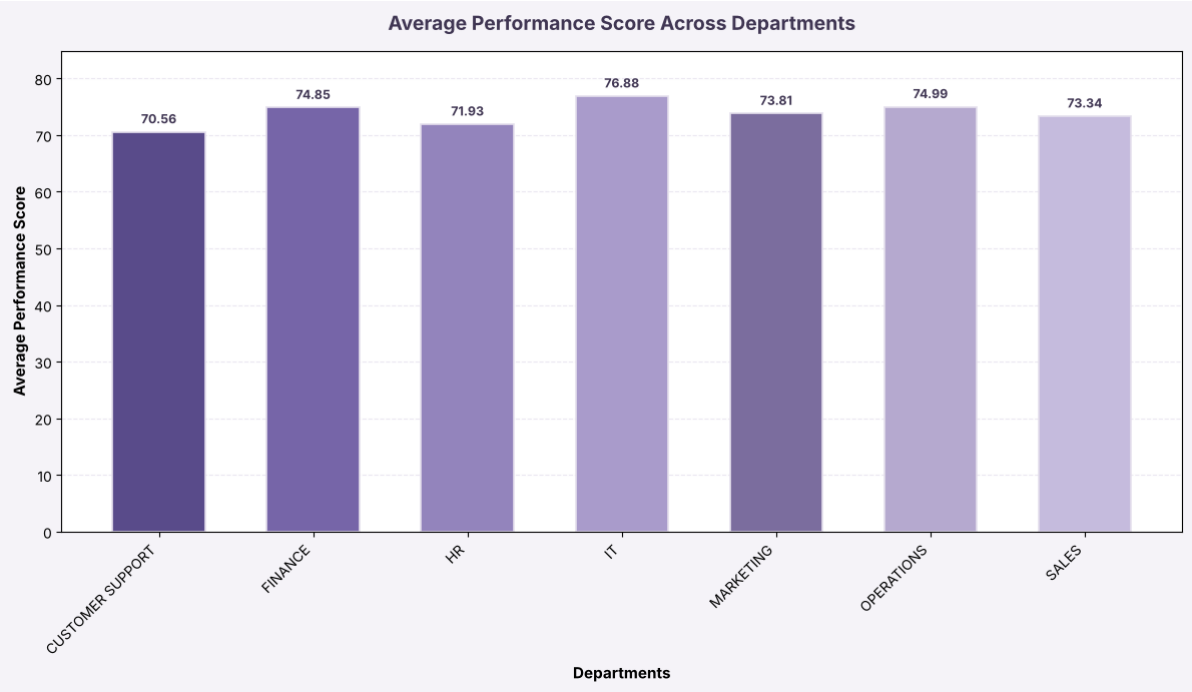
plt.xlabel("Departments", fontsize=11, fontweight="bold")
plt.ylabel("Average Performance Score", fontsize=11, fontweight="bold")

plt.xticks(rotation=45, ha="right")

plt.grid(axis="y", color="#E3DDEC", linestyle="--", alpha=0.7)
ax.set_axisbelow(True)

plt.tight_layout()
plt.show()
```

Screenshot: code for the bar chart



Screenshot: bar chart of average performance score by department

What the chart shows: IT has the highest average performance score (76.88), followed by Operations (74.99) and Finance (74.85). Customer Support has the lowest (70.56), followed by HR (71.93).

Task 7: Scatter Plot - Working Hours vs Projects Completed

Each dot is one employee. The plot shows whether employees who work more hours also complete more projects.

```
[53]:
hours = df["Working Hours"]
projects = df["Projects Completed"]

plt.figure(figsize=(10,6), facecolor="#F0FDF4")  # Figure background

plt.gca().set_facecolor("#ECFDF5")  # Plot background

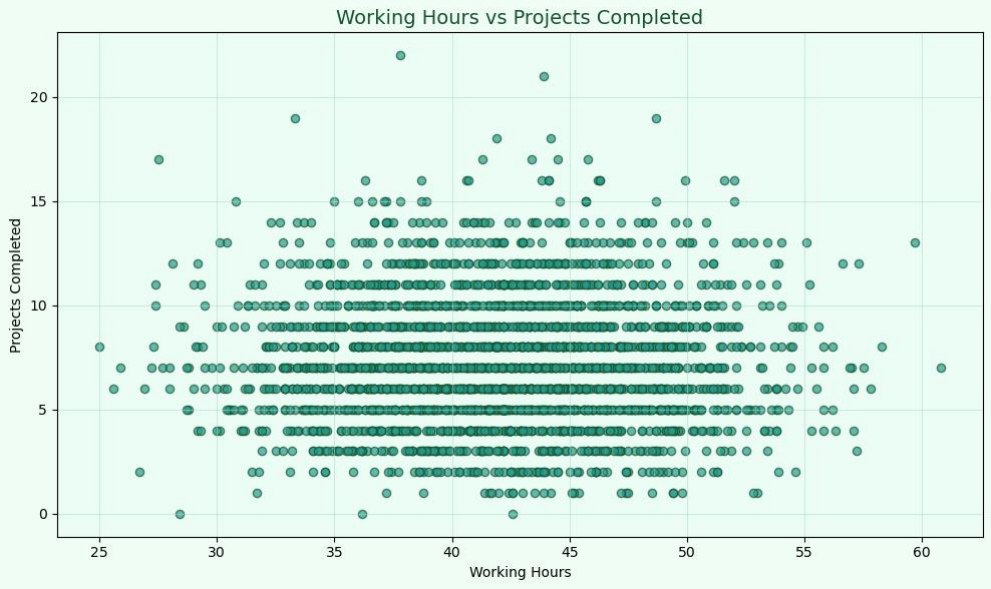
plt.scatter(hours, projects,
            color="#2A9D8F", s=35,
            marker="o", alpha=0.7,
            edgecolor="#14532D")  # Scatter points

plt.title("Working Hours vs Projects Completed", color="#14532D", fontsize=14)
plt.xlabel("Working Hours")
plt.ylabel("Projects Completed")

plt.grid(color="#A7DCCB", alpha=0.5)  # Grid color

plt.tight_layout() # Adjust layout
plt.show()
```

Screenshot: code for the scatter plot



Screenshot: scatter plot of working hours vs projects completed

What the chart shows: most employees work between about 32 and 52 hours and complete between 3 and 12 projects. The dots form a wide cloud with no upward or downward trend, so working more hours does not mean completing more projects. The few employees with very high project counts (17 to 22) work normal hours.

Task 8: Box Plot - Performance Scores across Designations

A box plot shows the middle value (median), the spread of the middle 50% of scores and possible outliers for each designation. It is used to compare designations fairly.

```
[61]:
ax = df.boxplot(
    column="Performance Score",
    by="Designation",
    figsize=(18,7),
    patch_artist=True,
    grid=False
)

# Apply different colors to each box
colors = ["#264653", "#2A9D8F", "#E9C46A", "#F4A261",
          "#E76F51", "#457B9D", "#8E7DBE", "#6A994E",
          "#D65DB1", "#00A6A6"]

for i, box in enumerate(ax.artists):
    box.set_facecolor(colors[i % len(colors)])
    box.set_edgecolor("#17365D")

plt.gcf().set_facecolor("#DCE6F1")
plt.gca().set_facecolor("#F4F8FC")

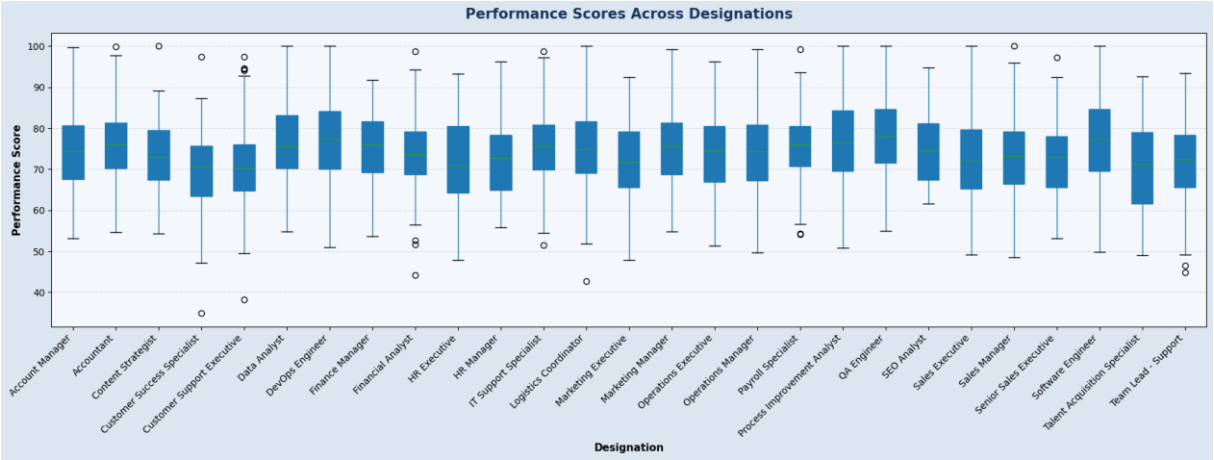
plt.title("Performance Scores Across Designations",
          fontsize=15, fontweight="bold", color="#17365D", pad=15)

plt.suptitle("")
plt.xlabel("Designation", fontsize=11, fontweight="bold")
plt.ylabel("Performance Score", fontsize=11, fontweight="bold")

plt.xticks(rotation=45, ha="right")
plt.grid(axis="y", linestyle="--", alpha=0.4)
plt.gca().set_axisbelow(True)

plt.tight_layout()
plt.show()
```

Screenshot: code for the box plot



Screenshot: box plot of performance scores by designation

What the chart shows: the median score of most designations lies between about 70 and 78. Designations such as QA Engineer, Software Engineer, DevOps Engineer and Process Improvement Analyst have some of the highest medians, while Customer Success Specialist and Customer Support Executive have some of the lowest. Customer Support Executive, Financial Analyst and Team Lead - Support show several low or high outliers (dots outside the whiskers).

Part 1 - Interpretation

Findings from the analysis

- **Departments:** the IT department has the highest average performance score (76.88), while Customer Support has the lowest (70.56). The gap between them is about 6.3 points.
- **Project completion:** all departments complete a similar number of projects (7.13 to 7.57 on average), but their performance scores differ more. So completing more projects does not always mean better performance; quality of work matters as much as quantity.
- **Working hours:** the scatter plot shows no clear link between working hours and projects completed. Longer working hours do not automatically give more output.
- **Designations:** performance differs across designations, with technical roles such as QA Engineer, Software Engineer and DevOps Engineer scoring higher and customer-facing support roles scoring lower.
- **Data quality:** the raw data had 12 spellings for 7 departments, invalid scores, a negative project count, a 95-hour working week, negative salaries and 5 duplicate rows. Cleaning them was necessary to get trustworthy results.

Overall, employee performance is different across departments and designations, while the number of projects and the working hours are almost the same everywhere.

Ways to Improve Employee Productivity

1. **Training:** provide proper training to employees to improve their skills.
2. **Listening:** ask employees about the problems they face while completing their tasks and try to solve them.
3. **Team leader:** appoint a team leader to guide employees whenever they need help.
4. **Balanced workload:** divide work equally among employees. If they feel tired due to too much work, give them enough rest.
5. **Employee of the Month:** start this program to motivate employees.
6. **Bonuses and rewards:** reward employees for their good work.
7. **Punctuality:** encourage employees to come on time so that projects can be completed on time.
8. **Work-life balance:** encourage employees to leave work on time and spend time with their families.
9. **Paid overtime:** if extra work is needed, ask employees to work overtime and pay them extra for those hours.
10. **Increments:** give salary increments to employees who complete their projects on time and maintain good quality.

Limitations

- Missing and invalid values were filled with the mean or median, which may slightly change the true distribution.
- The analysis is descriptive. It shows differences but does not prove cause and effect; for example, higher performance in IT may also be related to experience or role.
- Only the given dataset was used, and no correlation or regression test was applied.

Part 1 Conclusion

In this project the employee data was loaded, examined and cleaned. Inconsistent department names were standardised, invalid scores, working hours and salaries were corrected, missing values were filled with a suitable method, 5 duplicate rows were removed and Joining Date was converted to a proper date type.

The analysis then showed clear differences in performance across departments and designations, with IT performing best and Customer Support lowest. It also showed that completing more projects or working longer hours does not guarantee better performance.

Training, fair distribution of work, recognition, rewards and a healthy work-life balance can help the organisation improve employee productivity. In future, correlation and regression analysis, performance prediction models and an interactive HR dashboard can be added to make the study deeper.

PART 2: Employee Attrition and Retention Analysis

About Part 2

Employee attrition means employees leaving the company. Part 2 uses an employee file to find out how many employees left, which groups of employees leave more often, and what the company can do to keep its employees (retention).

Part 2 dataset: Task 2.csv - 2,678 rows and 9 columns.

Column	Meaning
Employee ID	Unique identification of each employee
Department	Department in which the employee works
Age	Age of the employee in years
Gender	Gender of the employee
Job Satisfaction	Satisfaction rating from 1 (lowest) to 5 (highest)
Salary	Salary received by the employee
Years at Company	Number of years the employee has worked in the company
Promotion History	Whether the employee has been promoted (Yes, No, Not Yet)
Attrition Status	Whether the employee left the company (YES) or stayed (NO)

Objectives of Part 2

- To clean the dataset and fix the inconsistent attrition labels.
- To calculate the overall employee attrition rate.
- To calculate attrition rates by department, age group and salary category.
- To study the relationship between years at the company and attrition.
- To visualise attrition and tenure with charts.
- To identify employee segments with high attrition and recommend retention strategies.

Quick summary of results (Part 2)

Item	Result
Records in raw file	2,678 rows x 9 columns (24,102 values)
Missing values found	310 (about 1.29% of all values)
Attrition labels	12 different spellings standardised into 2 labels: YES and NO
Employees who left / stayed	616 left (YES) and 2,062 stayed (NO)
Overall attrition rate	23.00% (77.00% of employees stayed)
Highest / lowest attrition department	Customer Support 27.61% / Finance 18.13%
Highest attrition by job satisfaction	Score 1: 46.30%, against only 7.87% for score 5
Lowest paying salary group	25.57% attrition (highest among salary groups)

Part 2 - Task 1: Clean the Dataset and Identify Inconsistent Attrition Labels

First, the libraries were imported and the CSV file was loaded into a Pandas DataFrame called `df`. The data was then examined for its size, data types, missing values and inconsistent labels, and every problem was fixed before the analysis.

1.1 Import libraries and load the data

```
[1]:  # Import libraries:
      import pandas as pd
      import numpy as np
      import matplotlib.pyplot as plt

[2]:  import pandas as pd
      df = pd.read_csv(r"C:\Users\vssag\Desktop\Task 2.csv")
```

Screenshot: importing libraries and loading the CSV file

1.2 First look at the data

`sample(5)`, `head()` and `tail()` show five random, the first five and the last five rows. Two problems are already visible: Gender has a value written as "unknown", and Attrition Status is written in many different ways, such as YES, Yes, Y, Active, Stayed, N and NO.

```
[3]:  df.sample(5) # It will give random 5 rows.
```

[3]:

	Employee ID	Department	Age	Gender	Job Satisfaction	Salary	Years at Company	Promotion History	Attrition Status
1813	EMP11814	HR	38.0	Male	3.0	50000.0	7.2	No	Left
718	EMP10719	Operations	38.0	Male	5.0	37500.0	4.0	No	NO
1292	EMP11293	Finance	53.0	Female	2.0	90500.0	9.5	Yes	N
1908	EMP11909	Marketing	36.0	Female	3.0	55500.0	8.2	Yes	STAYED
709	EMP10710	Finance	29.0	Female	4.0	59000.0	2.0	No	No

```
[4]:  df.head() # It shows top five rows
```

[4]:

	Employee ID	Department	Age	Gender	Job Satisfaction	Salary	Years at Company	Promotion History	Attrition Status
0	EMP10001	Operations	47.0	Male	2.0	53500.0	2.9	No	YES
1	EMP10002	IT	29.0	Female	4.0	59000.0	0.4	No	Yes
2	EMP10003	Marketing	29.0	Female	4.0	65000.0	9.6	No	Y
3	EMP10004	Operations	46.0	Female	4.0	57000.0	3.5	No	Active
4	EMP10005	Sales	36.0	unknown	1.0	50000.0	2.3	Yes	Stayed

```
[6]:  df.tail() # It shows top last rows
```

[6]:

	Employee ID	Department	Age	Gender	Job Satisfaction	Salary	Years at Company	Promotion History	Attrition Status
2673	EMP12674	Finance	37.0	Male	2.0	73500.0	1.6	Not Yet	Stayed
2674	EMP12675	Operations	53.0	Male	3.0	70500.0	8.4	Yes	Stayed
2675	EMP12676	Finance	35.0	Male	5.0	68500.0	5.3	No	STAYED
2676	EMP12677	Finance	46.0	Female	3.0	61000.0	1.8	Not Yet	N
2677	EMP12678	Marketing	28.0	Male	4.0	57500.0	1.8	Not Yet	NO

Screenshot: df.sample(5), df.head() and df.tail()

1.3 Column names, information and data types

The dataset has **2,678 rows and 9 columns**. Employee ID and Attrition Status have 2,678 values, but the other columns have fewer, which means they contain missing values. Age, Job Satisfaction, Salary and Years at Company are numbers (float64), and the other five columns are text (object).

```
[7]: print(df.columns.tolist()) # It shows All column names in a list.

['Employee ID', 'Department', 'Age', 'Gender', 'Job Satisfaction', 'Salary', 'Years at Company',
 'Promotion History', 'Attrition Status']
```

Screenshot: column names

```
[8]: df.info() # It will give Dataset structure.

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2678 entries, 0 to 2677
Data columns (total 9 columns):
 #   Column             Non-Null Count  Dtype
---  ------             --------------  -----
 0   Employee ID        2678 non-null   object
 1   Department         2646 non-null   object
 2   Age                2652 non-null   float64
 3   Gender             2630 non-null   object
 4   Job Satisfaction   2612 non-null   float64
 5   Salary             2638 non-null   float64
 6   Years at Company   2646 non-null   float64
 7   Promotion History  2612 non-null   object
 8   Attrition Status   2678 non-null   object
dtypes: float64(4), object(5)
memory usage: 188.4+ KB

[10]: df.dtypes # it will shows Data types of columns.

[10]: Employee ID          object
Department           object
Age                 float64
Gender               object
Job Satisfaction    float64
Salary              float64
Years at Company    float64
Promotion History    object
Attrition Status     object
dtype: object
```

Screenshot: df.info() and df.dtypes

1.4 Statistical summary

The `describe()` table shows that the numbers are in a sensible range: Age is from 21 to 60 (average 34.11), Job Satisfaction is from 1 to 5 (average 3.25), Salary is from 18,000 to 108,000 (average 58,258.53) and Years at Company is from 0.1 to 25 (average 4.62). No negative or impossible values were found, so only missing values and inconsistent labels needed to be fixed.

```
[9]: df.describe() # It shows statistical summary of the dataset
```

[9]:

	Age	Job Satisfaction	Salary	Years at Company
count	2652.000000	2612.000000	2638.000000	2646.000000
mean	34.110483	3.249234	58258.529189	4.624565
std	7.634488	1.134189	14563.666187	3.215006
min	21.000000	1.000000	18000.000000	0.100000
25%	28.000000	3.000000	48000.000000	2.300000
50%	34.000000	3.000000	57000.000000	3.900000
75%	39.000000	4.000000	68000.000000	6.100000
max	60.000000	5.000000	108000.000000	25.000000

Screenshot: df.describe()

1.5 Missing values

In total **310** values are missing: Job Satisfaction (66), Promotion History (66), Gender (48), Salary (40), Department (32), Years at Company (32) and Age (26). This is only about 1.29% of all values, so the data can be fixed instead of deleted. Employee ID and Attrition Status have no missing values.

```
[11]: df.isnull().any() # Check any missing value in dataset by each column.

[11]: Employee ID          False
      Department            True
      Age                   True
      Gender                True
      Job Satisfaction      True
      Salary                True
      Years at Company      True
      Promotion History     True
      Attrition Status     False
      dtype: bool

[12]: df.isnull().sum() # It shows all misiing not/nll in each column.

[12]: Employee ID           0
      Department           32
      Age                  26
      Gender               48
      Job Satisfaction     66
      Salary               40
      Years at Company     32
      Promotion History    66
      Attrition Status      0
      dtype: int64
```

Screenshot: missing values in each column

1.6 Unique labels before cleaning

The unique labels of Department, Gender, Promotion History and Attrition Status were printed. Gender contains " unknown " with extra spaces, and Attrition Status has **12 different labels** that really mean only two things: the employee left or the employee stayed.

```
[13]:  df["Department"].unique()        # Check department labels

[13]:  array(['Operations', 'IT', 'Marketing', 'Sales', 'Customer Support',
              'Finance', 'HR', nan], dtype=object)

[14]:  df["Gender"].unique()            # Check gender labels

[14]:  array(['Male', 'Female', ' unknown ', nan], dtype=object)

[15]:  df["Promotion History"].unique() # Check promotion labels

[15]:  array(['No', 'Yes', 'Not Yet', nan], dtype=object)

[16]:  df["Attrition Status"].unique()  # Check attrition labels

[16]:  array(['YES ', 'Yes', 'Y', 'Active', 'Stayed', 'STAYED', 'NO ', 'N',
              'Left', 'No', 'LEFT', 'Resigned'], dtype=object)
```

Screenshot: unique labels of Department, Gender, Promotion History and Attrition Status

1.7 Summary: how every column was handled

Before fixing anything, one rule was followed: never fill a missing value in a way that creates false information. Text columns where the value cannot be guessed (Gender, Promotion History) were filled with "Unknown". Department is a category, so its most common value (mode) was used. For numbers, the median was used because it is not pulled by extreme values.

Column	Missing	Problem found	How it was handled	Why
Department	32	Labels in different forms; IT and HR need standard names	Trimmed, converted to title case, "Hr" changed to "Human Resources"; missing filled with the mode (Operations)	A department is a category; the most common value is the best guess
Age	26	None	Filled with the median (34)	Median ignores extreme ages
Gender	48	" unknown " with extra spaces	Trimmed, title case; missing filled with "Unknown"	Gender cannot be guessed
Job Satisfaction	66	None	Filled with the median (3)	Rating is 1 to 5; the median is a whole-number rating
Salary	40	None	Filled with the median (57,000)	Median ignores very high salaries
Years at Company	32	None	Filled with the median (3.9)	Tenure is skewed, so the median is safer
Promotion History	66	Spaces and case differences	Trimmed, title case; missing filled with "Unknown"	Promotion cannot be guessed
Attrition Status	0	12 different labels	Standardised into YES and NO	Only two outcomes are possible

1.8 Department: inconsistent labels

Text was trimmed and converted to title case, and "Hr" was replaced with "Human Resources". IT is shown as "It" because of the title-case conversion; it is still one single IT group, so the results are not affected.

```
[18]: df["Department"] = df["Department"].str.strip().str.title().replace({  # Correct inconsistent department names
          "Hr": "Human Resources",
          "I.T.": "IT"
      })

[19]: df["Department"].unique()  # Check department names again

[19]: array(['Operations', 'It', 'Marketing', 'Sales', 'Customer Support',
             'Finance', 'Human Resources', nan], dtype=object)
```

Screenshot: Department labels after cleaning

1.9 Gender and Promotion History

Extra spaces were removed and the text was converted to title case, so " unknown " became "Unknown". The same was done for Promotion History, which has the labels No, Yes and Not Yet.

```
[20]: df["Gender"] = df["Gender"].str.strip().str.title()  # Correct inconsistent gender values

[21]: df["Gender"].unique()  # Check unique values again

[21]: array(['Male', 'Female', 'Unknown', nan], dtype=object)
```

Screenshot: Gender after cleaning

```
[22]: df["Promotion History"] = df["Promotion History"].str.strip().str.title()  # Correct inconsistent values

[23]: df["Promotion History"].unique()  # Check unique values again

[23]: array(['No', 'Yes', 'Not Yet', nan], dtype=object)
```

Screenshot: Promotion History after cleaning

1.10 Attrition Status: standardising the labels

This is the most important cleaning step. The labels were trimmed and converted to upper case, then the different words were grouped into two labels:

Original labels	Standard label	Meaning
YES, Y, LEFT, RESIGNED	YES	The employee left the company
NO, N, STAYED, ACTIVE	NO	The employee is still in the company

```
[24]: df["Attrition Status"] = df["Attrition Status"].str.strip().str.upper().replace({  # Standardize attrition values
          "Y": "YES",
          "LEFT": "YES",
          "RESIGNED": "YES",
          "N": "NO",
          "STAYED": "NO",
          "ACTIVE": "NO"
      })

[25]: df["Attrition Status"].unique()  # Check unique values again

[25]: array(['YES', 'NO'], dtype=object)
```

Screenshot: Attrition Status before and after standardising

1.11 Filling the missing values

Each column was filled with its own method and checked again with `isnull().sum()`, which shows 0 after filling. Department was filled with the mode, which is Operations.

```
[27]: df["Department"].mode()  # Find most frequent department

[27]: 0    Operations
      Name: Department, dtype: object

[28]: df["Department"] = df["Department"].fillna(df["Department"].mode()[0])  # Fill missing values with mode
```

Screenshot: Department: most common value and filling

Age was filled with the median.

```
[29]: df["Age"].describe()  # Check age Statistical summary.

[29]: count    2652.000000
      mean       34.110483
      std         7.634488
      min        21.000000
      25%        28.000000
      50%        34.000000
      75%        39.000000
      max        60.000000
      Name: Age, dtype: float64

[30]: df["Age"] = df["Age"].fillna(df["Age"].median())  # Fill missing values with median
```

Screenshot: Age: summary and filling

Gender was filled with "Unknown". Before filling, there were 1,516 Male, 1,102 Female and 12 Unknown employees.

```
[31]: df["Gender"].value_counts()  # Check gender distribution

[31]: Gender
      Male       1516
      Female     1102
      Unknown      12
      Name: count, dtype: int64

[32]: df["Gender"] = df["Gender"].fillna("Unknown")  # Fill missing gender values

[33]: df["Gender"].isnull().sum()  # Check missing values again

[33]: np.int64(0)
```

Screenshot: Gender: distribution and filling

Job Satisfaction was filled with the median (3).

```
[34]: df["Job Satisfaction"].describe()  # Check statistical distribution

[34]: count    2612.000000
      mean        3.249234
      std         1.134189
      min         1.000000
      25%         3.000000
      50%         3.000000
      75%         4.000000
      max         5.000000
      Name: Job Satisfaction, dtype: float64

[35]: df["Job Satisfaction"] = df["Job Satisfaction"].fillna(df["Job Satisfaction"].median())  # Fill missing values with median

[36]: df["Job Satisfaction"].isnull().sum()  # Check missing values again

[36]: np.int64(0)
```

Screenshot: Job Satisfaction: summary and filling

Salary was filled with the median (57,000).

```
[37]: df["Salary"].describe()  # Check statistical distribution

[37]: count      2638.000000
      mean      58258.529189
      std       14563.666187
      min       18000.000000
      25%       48000.000000
      50%       57000.000000
      75%       68000.000000
      max      108000.000000
      Name: Salary, dtype: float64

[38]: df["Salary"] = df["Salary"].fillna(df["Salary"].median())  # Fill missing values with median

[39]: df["Salary"].isnull().sum()  # Check missing values again

[39]: np.int64(0)
```

Screenshot: Salary: summary and filling

Years at Company was filled with the median (3.9).

```
[40]: df["Years at Company"].describe()  # Check statistical distribution

[40]: count    2646.000000
      mean        4.624565
      std         3.215006
      min         0.100000
      25%         2.300000
      50%         3.900000
      75%         6.100000
      max        25.000000
      Name: Years at Company, dtype: float64

[41]: df["Years at Company"] = df["Years at Company"].fillna(df["Years at Company"].median())  # Fill missing values with median

[42]: df["Years at Company"].isnull().sum()  # Check missing values again

[42]: np.int64(0)
```

Screenshot: Years at Company: summary and filling

Promotion History was filled with "Unknown". Attrition Status had no missing values, so the same step changed nothing there.

```
[43]: df["Promotion History"] = df["Promotion History"].fillna("Unknown")  # Replace missing values with Unknown

[44]: df["Promotion History"].isnull().sum()  # Check missing values again

[44]: np.int64(0)

[45]: df["Attrition Status"] = df["Attrition Status"].fillna("Unknown")  # Replace missing values with Unknown

[46]: df["Attrition Status"].isnull().sum()  # Check missing values again

[46]: np.int64(0)
```

Screenshot: Promotion History and Attrition Status: filling

1.12 Final check

After cleaning, every column shows 0 missing values and the dataset is ready for analysis.

```
[47]: df.isnull().sum()  # Check remaining missing values

[47]: Employee ID          0
      Department           0
      Age                  0
      Gender               0
      Job Satisfaction     0
      Salary               0
      Years at Company     0
      Promotion History    0
      Attrition Status     0
      dtype: int64
```

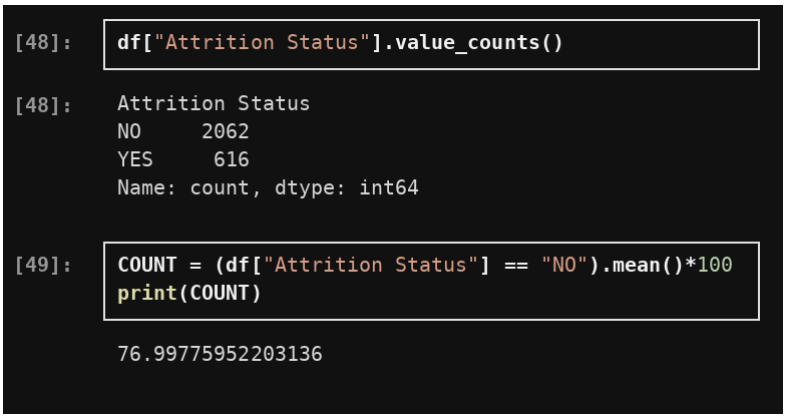
Screenshot: missing values after cleaning

Part 2 - Task 2: Calculate the Overall Employee Attrition Rate

`value_counts()` shows that **616 employees left (YES)** and **2,062 employees stayed (NO)**, out of 2,678 employees.

The code `(df["Attrition Status"] == "NO").mean() * 100` gives 76.998, which is the percentage of employees who **stayed** (the retention rate). The attrition rate is the percentage of employees who left, so it is the rest:

Measure	Calculation	Result
Retention rate (stayed)	2,062 / 2,678 x 100	77.00%
Attrition rate (left)	616 / 2,678 x 100 = 100 - 77.00	23.00%



Screenshot: number of employees who left and stayed, and the percentage who stayed

About 23 out of every 100 employees leave the company.

Part 2 - Task 3: Attrition Rates by Department, Age Group and Salary Category

`pd.crosstab(..., normalize="index")` was used to find, inside each group, the share of employees who stayed or left. This allows a fair comparison between groups of different sizes.

3.1 By department

The crosstab below shows the percentage of employees who **stayed** in each department. The attrition rate is 100 minus this value (it is also calculated directly in Task 6).

```
[50]: pd.crosstab(df['Department'], df['Attrition Status'], normalize='index')['NO'] * 100

[50]: Department
      Customer Support    72.390572
      Finance             81.873112
      Human Resources     74.162679
      It                  79.157895
      Marketing           74.349442
      Operations          79.965753
      Sales               73.684211
      Name: NO, dtype: float64
```

Screenshot: percentage of employees who stayed in each department

Department	Stayed (%)	Left - attrition rate (%)
Customer Support	72.39	27.61
Sales	73.68	26.32
Human Resources	74.16	25.84
Marketing	74.35	25.65
IT (shown as "It")	79.16	20.84
Operations	79.97	20.03
Finance	81.87	18.13

3.2 By age group

Employees were divided into three age groups: **Young** (below 30), **Middle** (30 to 39) and **Senior** (40 and above). The result is the attrition rate (YES) of each group.

```
[51]: df['Age Group'] = 'Senior'
      df.loc[df['Age'] < 40, 'Age Group'] = 'Middle'
      df.loc[df['Age'] < 30, 'Age Group'] = 'Young'

      pd.crosstab(df['Age Group'], df['Attrition Status'], normalize='index')["YES"] * 100

[51]: Age Group
      Middle    21.840000
      Senior    23.987539
      Young     24.045802
      Name: YES, dtype: float64
```

Screenshot: attrition rate by age group

Young employees (24.05%) and senior employees (23.99%) leave a little more than middle-aged employees (21.84%).

3.3 By salary category

Employees were divided into three salary groups: **Lowest paying** (18,000 to 50,000), **Mid paying** (50,000 to 95,000) and **Highest paying** (above 95,000). The group names in the screenshot contain small spelling mistakes ("Highesh", "Employess"), but the groups are the same.

```
[52]: df["Salary Group"]="Highesh Paying Employees"
      df.loc[df["Salary"].between(18000,50000),"Salary Group"] = "Lowest Paying Employess"
      df.loc[df["Salary"].between(50000,95000),"Salary Group"] = "Mid Paying Employees"


      pd.crosstab(df["Salary Group"],df["Attrition Status"],normalize='index')["YES"]*100

[52]: Salary Group
      Highesh Paying Employees    23.809524
      Lowest Paying Employess     25.566751
      Mid Paying Employees        21.900161
      Name: YES, dtype: float64
```

Screenshot: attrition rate by salary category

The lowest paying group has the highest attrition (25.57%), the mid paying group has the lowest (21.90%), and the highest paying group is in between (23.81%).

Part 2 - Task 4: Relationship between Years at the Company and Attrition

Employees were grouped by their years at the company and the attrition rate (YES) of every group was calculated. The groups were created in the code like this:

Group name in the code	Years at Company	Attrition rate
Experienced Employees	2 years or less	26.25%
Mid Level Experienced Employees	3 to 5 years	23.03%
Fresher Employees	Between 2 and 3 years, and above 5 years	21.66%



```
[53]: df["Years at Company Group"] = "Experienced Employees"
      df.loc[df["Years at Company"] > 2,"Years at Company Group"] ="Fresher Employees"
      df.loc[df["Years at Company"].between(3,5),"Years at Company Group"] = "Mid Level Experienced Employees"


      pd.crosstab(df["Years at Company Group"],df["Attrition Status"],normalize='index')['YES']*100

[53]: Years at Company Group
      Experienced Employees             26.247689
      Fresher Employees                 21.660377
      Mid Level Experienced Employees   23.029557
      Name: YES, dtype: float64
```

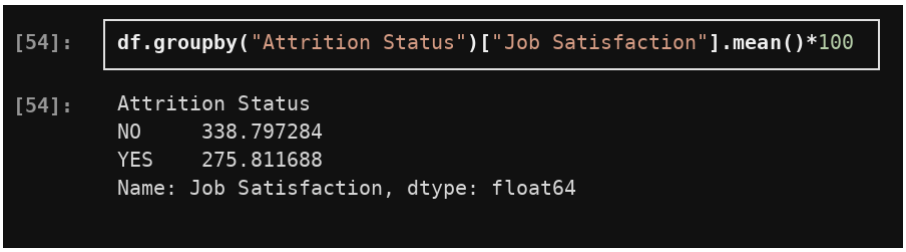
Screenshot: attrition rate by years at company group

Note on the group names: in the code, the default group (2 years or less) was named "Experienced Employees", while longer tenures were named "Fresher Employees". The names are therefore the other way round, but the numbers are correct for the year ranges given in the table above.

Finding: employees with 2 years or less in the company have the highest attrition rate (26.25%), so new employees are the most likely to leave. Employees with 3 to 5 years leave a little less (23.03%), and the lowest rate (21.66%) is in the remaining group.

Part 2 - Task 5: Job Satisfaction of Employees who Stayed and Left

The average Job Satisfaction was calculated separately for employees who stayed (NO) and those who left (YES). The code multiplies the average by 100, so the output shows 338.80 and 275.81. In the original 1 to 5 scale these are **3.39** and **2.76**.



Screenshot: average job satisfaction by attrition status (multiplied by 100)

Attrition Status	Average Job Satisfaction (1 to 5)
Stayed (NO)	3.39
Left (YES)	2.76

Employees who left were clearly less satisfied with their job than those who stayed.

Part 2 - Visualization Tasks

Three charts were created: a bar chart of attrition by department, a histogram of employee tenure and a stacked bar chart of attrition across job satisfaction levels.

Task 6: Bar Chart - Department-wise Attrition Rates

First the attrition rate (YES) of each department was calculated, then it was shown as a bar chart with the percentage printed above every bar.

[55]:
```
# Calculate department-wise attrition rate

dept_attrition = df.groupby("Department")["Attrition Status"].value_counts(normalize=True).unstack() * 100

dept_attrition = dept_attrition["YES"]  # Get attrition rate
print(dept_attrition)
```

```
Department
Customer Support    27.609428
Finance             18.126888
Human Resources     25.837321
It                  20.842105
Marketing           25.650558
Operations          20.034247
Sales               26.315789
Name: YES, dtype: float64
```

Screenshot: department-wise attrition rate

[59]:
```
plt.figure(figsize=(10, 7), facecolor="#F3F6F9")
ax = plt.gca()
ax.set_facecolor("#FFFFFF")

colors = ["#365F91", "#527DAA", "#6A91B5", "#78A6A6",
          "#8FAFC4", "#648C9E", "#A3B8C8"]

bars = plt.bar(dept_attrition.index, dept_attrition.values,
               color=colors[:len(dept_attrition)],
               edgecolor="#FFFFFF", linewidth=1.2, width=0.6)

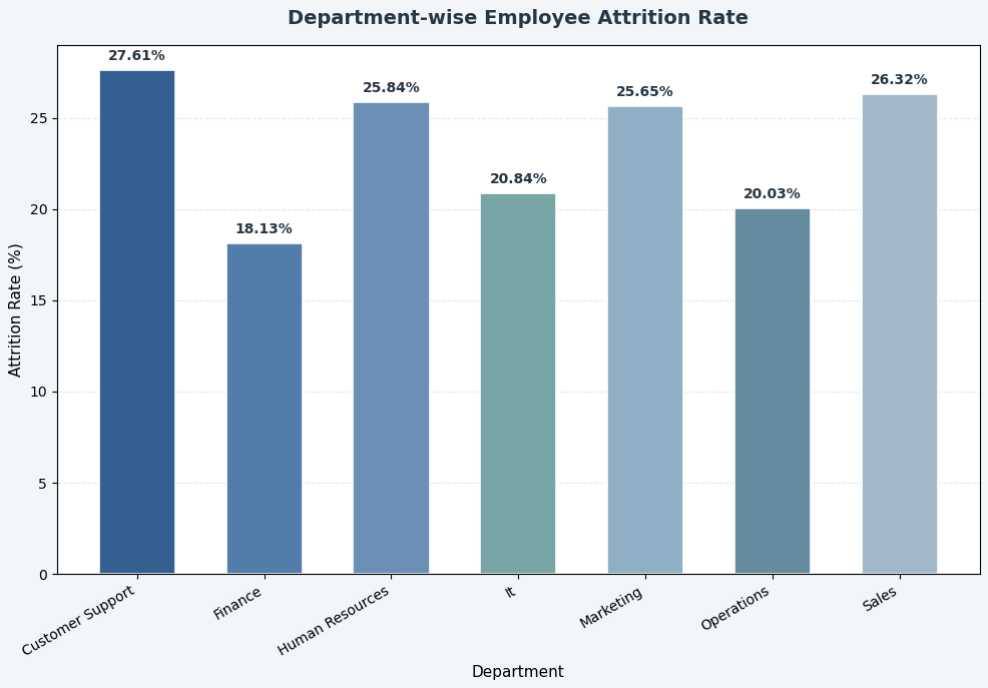
plt.title("Department-wise Employee Attrition Rate",
          fontsize=14, fontweight="bold", color="#263746", pad=15)
plt.xlabel("Department", fontsize=11)
plt.ylabel("Attrition Rate (%)", fontsize=11)

plt.bar_label(bars, fmt="%.2f%%", padding=5,
              fontsize=10, fontweight="bold", color="#263746")  # Show values above bars

plt.grid(axis="y", linestyle="--", color="#D9E1E8", alpha=0.7)
ax.set_axisbelow(True)

plt.xticks(rotation=30, ha="right")
plt.tight_layout()
plt.show()
```

Screenshot: code for the bar chart



Screenshot: bar chart of department-wise employee attrition rate

What the chart shows: Customer Support has the highest attrition rate (27.61%), followed by Sales (26.32%), Human Resources (25.84%) and Marketing (25.65%). Finance has the lowest (18.13%), followed by Operations (20.03%) and IT (20.84%).

Task 7: Histogram - Employee Tenure

The histogram shows how many employees have worked for a given number of years. The number of employees is printed above each bar.

```
[60]:
employee = df["Years at Company"]  #

plt.figure(figsize=(9, 6), facecolor="#F5F3F8")  # Set chart size and ba
ax = plt.gca()  # Get current axes
ax.set_facecolor("#FFFFFF")  # Set plot background

counts, bins, patches = plt.hist(
    employee, bins=12,  # Divide data into 12 intervals
    color="#8276A8",
    edgecolor="#FFFFFF",
    linewidth=1.2,
    alpha=0.9
)

plt.title("Distribution of Employee Tenure",
          fontsize=14, fontweight="bold", color="#403653", pad=15)

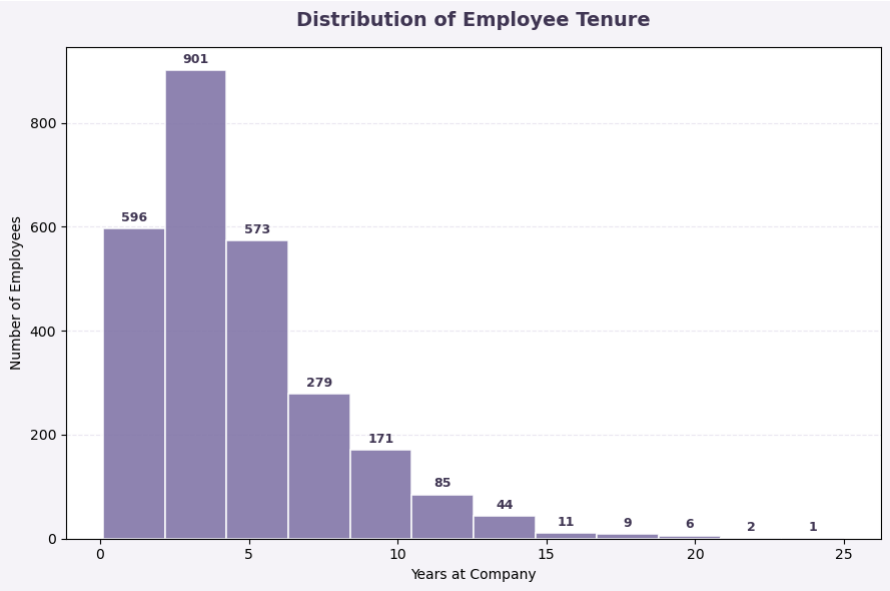
plt.xlabel("Years at Company")
plt.ylabel("Number of Employees")

plt.bar_label(
    patches,
    labels=[str(int(x)) if x > 0 else "" for x in counts],
    padding=3, fontsize=9, fontweight="bold",
    color="#403653"
)  # Display employee counts on bars

plt.grid(axis="y", linestyle="--", color="#E3DDEC", alpha=0.7)  # Add grid lines
ax.set_axisbelow(True)  # Keep grid behind bars

plt.tight_layout()  # Adjust chart spacing
plt.show()  # Display chart
```

Screenshot: code for the histogram



Screenshot: distribution of employee tenure

What the chart shows: most employees are fairly new. The highest bar is at about 2 to 4 years (901 employees), and 2,070 of 2,678 employees (about 77%) have worked around 6 years or less. Only 29 employees have more than about 15 years of service.

Task 8: Stacked Bar Chart - Attrition across Job Satisfaction

For every job satisfaction level, the bar shows how many employees stayed (NO) and how many left (YES).

```
[58]:
left = df[df["Attrition Status"] == "YES"].groupby("Job Satisfaction").size()  # Count employees who left
stayed = df[df["Attrition Status"] == "NO"].groupby("Job Satisfaction").size()  # Count employees who stayed

data = pd.DataFrame({"NO": stayed, "YES": left}).fillna(0)  # Combine both counts

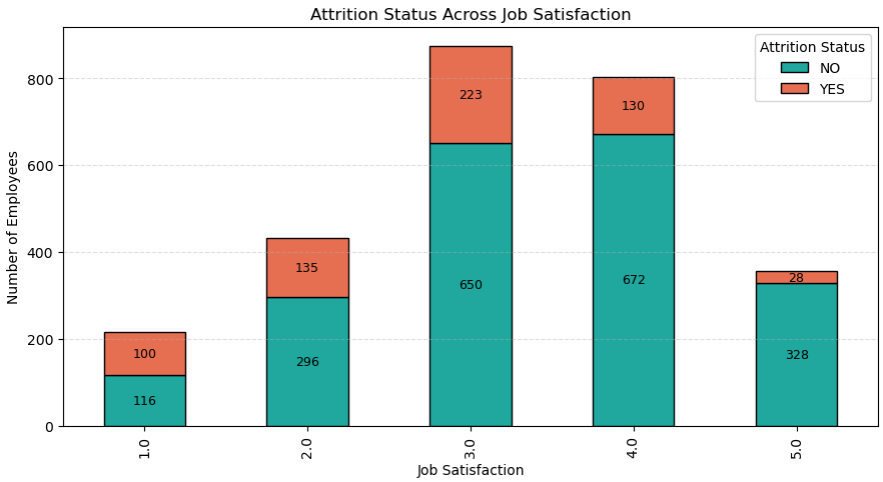
ax = data.plot.bar(stacked=True, figsize=(9, 5),
                   color=["#20A89E", "#E76F51"], edgecolor="black")

plt.xlabel("Job Satisfaction")
plt.ylabel("Number of Employees")
plt.title("Attrition Status Across Job Satisfaction")
plt.legend(title="Attrition Status")
plt.grid(axis="y", linestyle="--", alpha=0.4)

for container in ax.containers:
    ax.bar_label(container, label_type="center", fontsize=9)  # Show employee counts

plt.tight_layout()
plt.show()
```

Screenshot: code for the stacked bar chart



Screenshot: attrition status across job satisfaction

What the chart shows: the red part (employees who left) is large at low satisfaction and very small at high satisfaction. Using the counts printed in the chart, the attrition rate of each level is:

Job Satisfaction	Stayed (NO)	Left (YES)	Total	Attrition rate
1	116	100	216	46.30%
2	296	135	431	31.32%
3	650	223	873	25.54%
4	672	130	802	16.21%
5	328	28	356	7.87%

The attrition rate falls steadily as job satisfaction rises, from 46.30% at level 1 to only 7.87% at level 5.

Part 2 - Interpretation

Task 9: Employee Segments with High Attrition

Different groups of employees were compared with the overall attrition rate of **23.00%** to find which employees leave more often.

Segment	Attrition rate	Comment
Job Satisfaction score 1	46.30%	Highest of all; almost 1 in 2 leaves
Job Satisfaction score 2	31.32%	Also far above the overall rate
Department: Customer Support	27.61%	Highest department
Promotion History: No promotion	26.40%	Higher than employees who were promoted
Tenure: 2 years or less	26.25%	Newest employees leave most
Salary: Lowest paying group	25.57%	Highest salary group
Age: Young (below 30)	24.05%	Higher than middle-aged employees
Gender: Male	23.55%	Slightly higher than female employees

Conclusion: employees with low job satisfaction, lower salaries and no promotion, those who are new to the company and those who work in Customer Support are more likely to leave. Job satisfaction has by far the strongest link with attrition.

Task 10: Strategies to Reduce Attrition

1. **Low salary:** increase the salary of low paid employees so that they do not leave because of pay.
2. **Unhappy with the job:** ask employees why they are not happy and try to solve their problems.
3. **Promotions:** give promotions at the right time so that employees feel valued and see a chance to grow.
4. **Customer Support:** give employees proper breaks, because the job can be stressful.
5. **Young and new employees:** give proper training so that they understand their work and can do it better.
6. **Rewards:** reward employees or give bonuses when they do well.
7. **Flexible hours:** give employees some flexibility with their working hours.
8. **Skill development:** provide training so employees can learn new skills and improve their performance.
9. **Work environment:** keep a friendly workplace where employees feel comfortable.
10. **Manager support:** managers should listen to employees and help them when they face problems at work.

Limitations

- Missing values were filled with the median, mode or "Unknown", which may slightly change the true picture.
- The analysis is descriptive. It shows which groups leave more but does not prove why they leave.
- The results depend on how the age, salary and tenure groups were defined; different group limits may give slightly different rates.

Part 2 Conclusion

In Part 2 the attrition data was cleaned: 12 different attrition labels were standardised into YES and NO, department, gender and promotion labels were corrected and 310 missing values were filled. The analysis showed that **23.00%** of employees (616 out of 2,678) left the company.

Attrition was highest in Customer Support (27.61%) and lowest in Finance (18.13%). It was also higher among employees with low salaries, no promotion, young age and short tenure. The strongest pattern was job satisfaction: 46.30% of employees with a score of 1 left, compared with only 7.87% of those with a score of 5.

To keep employees, the company should improve job satisfaction, pay fair salaries, give promotions on time, support new and young employees with training, and reduce stress in Customer Support. In future, correlation analysis and a prediction model for attrition can be added to find employees at risk before they leave.

PART 3: Recruitment and Hiring Funnel Analysis

About Part 3

A hiring funnel shows how candidates move from one recruitment stage to the next: applying, screening, interview, offer and joining. At every stage some candidates are lost. Part 3 uses a recruitment file to count the candidates at each stage, calculate conversion rates, compare recruitment sources, measure the hiring time and find the stage where most candidates are lost.

Part 3 dataset: Task 3.csv - 2,930 rows and 10 columns.

Column	Meaning
Candidate ID	Unique identification of each candidate
Job Role	Job role for which the candidate applied
Application Date	Date on which the candidate applied
Screening Status	Result of screening (Passed, Rejected, Pending)
Interview Score	Score given to the candidate in the interview
Interview Status	Result of the interview (Selected, Rejected, Pending, Not Scheduled)
Offer Status	Result of the job offer (Accepted, Declined, Pending, Not Offered)
Joining Status	Whether the candidate joined (Joined, Not Joined, Withdrawn, Pending)
Recruitment Source	Channel through which the candidate was found (LinkedIn, Naukri, Indeed, etc.)
Joining Date	Date on which the candidate joined the company

Objectives of Part 3

- To clean the candidate records and standardise the recruitment stage labels.
- To calculate the number of candidates at each recruitment stage.
- To calculate the screening, interview, offer acceptance and joining conversion rates.
- To compare recruitment sources based on candidate conversion rates.
- To calculate the average hiring time (days from application to joining).
- To visualise the funnel, the conversion rates of sources and the turnaround time by job role.
- To find where most candidates are lost and to suggest ways to improve the hiring process.

Quick summary of results (Part 3)

Item	Result
Records in raw file	2,930 rows x 10 columns (29,300 values)
Missing values found	3,444 (Joining Date 2,298, Interview Score 1,137, Recruitment Source 5, Job Role 4)
Inconsistent labels	Spelling, capital letters and extra spaces fixed in Screening, Interview, Offer and Joining Status and in Recruitment Source
Rows removed: exact duplicates	6
Records after cleaning	2,924 rows x 10 columns
Candidates at each stage	2,053 passed screening, 1,150 selected in interview, 707 accepted the offer, 631 joined
Conversion rates (of 2,924 applicants)	Screening 70.21%, interview 39.33%, offer acceptance 24.18%, joining 21.58%

Item	Result
Best / lowest recruitment source	Employee Referral 84.02% / Campus Recruitment 62.10% (screening pass rate)
Average hiring time	34.74 days (for candidates who joined)
Biggest loss of candidates	Between screening and interview: 903 candidates

Part 3 - Task 1: Load the Dataset and Clean Candidate Records

First, the libraries were imported and the CSV file was loaded into a Pandas DataFrame called df. The data was then examined for its size, data types, missing values and inconsistent labels, and the problems were fixed before the analysis.

1.1 Import libraries and load the data

Pandas is used for tables and calculations, NumPy for numbers, and Matplotlib for charts.

```
[1]:
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

[2]:
df = pd.read_csv(r"C:\Users\vssag\Desktop\Task 3.csv")
```

Screenshot: importing libraries and loading the CSV file

1.2 First look at the data

sample(5) shows 5 random rows, head() shows the first five rows and tail() shows the last five rows. Each row is one candidate. Candidates who were rejected early have NaN (empty) values in Interview Score and Joining Date, because they never reached those stages.

```
[3]:
df.sample(5) # It will give random 5 rows.
```

[3]:

	Candidate ID	Job Role	Application Date	Screening Status	Interview Score	Interview Status	Offer Status	Joining Status	Recruitment Source	Joining Date
1778	CAND-01782	Software Engineer	2023-05-01	Passed	4.0	Rejected	Not Offered	Not Joined	Indeed	NaN
2334	CAND-02339	Project Manager	2025-10-15	Rejected	NaN	Not Scheduled	Not Offered	Not Joined	Company Website	NaN
2133	CAND-02137	Software Engineer	2025-11-12	Passed	NaN	Not Scheduled	Not Offered	Not Joined	Employee Referral	NaN
1979	CAND-01983	Marketing Specialist	2025-09-28	Passed	NaN	Not Scheduled	Not Offered	Not Joined	Naukri	NaN
1484	CAND-01487	Sales Executive	2023-04-03	Passed	7.2	Selected	Accepted	Joined	Indeed	2023-05-27

```
[4]:
df.head() # It shows top five rows
```

[4]:

	Candidate ID	Job Role	Application Date	Screening Status	Interview Score	Interview Status	Offer Status	Joining Status	Recruitment Source	Joining Date
0	CAND-00001	Sales Executive	2024-08-21	Passed	6.2	Selected	Accepted	Joined	Employee Referral	2024-09-09
1	CAND-00002	Customer Support	2024-07-04	Rejected	NaN	Not Scheduled	Not Offered	Not Joined	Naukri	NaN
2	CAND-00003	Software Engineer	2025-04-22	Rejected	NaN	Not Scheduled	Not Offered	Not Joined	Naukri	NaN
3	CAND-00004	Software Engineer	2023-01-09	Passed	5.8	Selected	Accepted	Joined	Employee Referral	2023-02-12
4	CAND-00005	Software Engineer	2023-09-21	Rejected	NaN	Not Scheduled	Not Offered	Not Joined	Recruitment Agency	NaN

```
[6]:
df.tail() # It shows top last rows
```

[6]:

	Candidate ID	Job Role	Application Date	Screening Status	Interview Score	Interview Status	Offer Status	Joining Status	Recruitment Source	Joining Date
2925	CAND-00552	Project Manager	2023-11-21	Passed	5.0	Pending	Not Offered	Not Joined	LinkedIn	NaN
2926	CAND-02784	Finance Analyst	2024-02-16	Passed	7.5	Selected	Accepted	Joined	Naukri	2024-05-06
2927	CAND-01894	HR Executive	2025-07-23	Passed	NaN	Not Scheduled	Not Offered	Not Joined	LinkedIn	NaN
2928	CAND-00934	Data Analyst	2023-04-21	Rejected	NaN	Not Scheduled	Not Offered	Not Joined	Company Website	NaN
2929	CAND-02540	Customer Support	2023-04-04	Rejected	NaN	Not Scheduled	Not Offered	Not Joined	LinkedIn	NaN

Screenshot: df.sample(5), df.head() and df.tail()

1.3 Column names, information and data types

The dataset has **2,930 rows and 10 columns**. The info() output shows that Interview Score is a decimal number (float64) and all other columns, including Application Date and Joining Date, are stored as plain text (object). Interview Score has only 1,793 filled values and Job Role has 2,926.

```
[7]: print(df.columns.tolist()) # It shows All column names in a list.

['Candidate ID', 'Job Role', 'Application Date', 'Screening Status', 'Interview Score',
    'Interview Status', 'Offer Status', 'Joining Status', 'Recruitment Source', 'Joining Date']

[8]: df.info() # It will give Dataset structure.

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2930 entries, 0 to 2929
Data columns (total 10 columns):
 #   Column              Non-Null Count  Dtype
---  ------              --------------  -----
 0   Candidate ID        2930 non-null   object
 1   Job Role            2926 non-null   object
 2   Application Date    2930 non-null   object
 3   Screening Status    2930 non-null   object
 4   Interview Score     1793 non-null   float64
 5   Interview Status    2930 non-null   object
 6   Offer Status        2930 non-null   object
 7   Joining Status      2930 non-null   object
 8   Recruitment Source  2925 non-null   object
 9   Joining Date        632 non-null    object
dtypes: float64(1), object(9)
memory usage: 229.0+ KB
```

Screenshot: column names and df.info()

1.4 Statistical summary

Interview Score is the only numeric column. Its average is 6.48, the middle value (median) is 6.5 and the standard deviation is 1.53. The lowest score is 0 and the highest is 11.

```
[9]: df.describe() # It shows statistical summary of the dataset
```

[9]:

	Interview Score
count	1793.000000
mean	6.479922
std	1.532807
min	0.000000
25%	5.400000
50%	6.500000
75%	7.600000
max	11.000000

Screenshot: df.describe()

1.5 Missing values

isnull().any() shows the columns that have missing values, and isnull().sum() counts them: Joining Date 2,298, Interview Score 1,137, Recruitment Source 5 and Job Role 4, which is **3,444** missing values in total. Most of them are not errors. A candidate who was not interviewed has no interview score, and a candidate who did not join has no joining date.

```
[10]: df.isnull().any() # Check any missing value in dataset by each column.

[10]: Candidate ID          False
      Job Role               True
      Application Date      False
      Screening Status      False
      Interview Score        True
      Interview Status      False
      Offer Status          False
      Joining Status        False
      Recruitment Source     True
      Joining Date           True
      dtype: bool

[11]: df.isnull().sum() # It shows all misiing not/nll in each column.

[11]: Candidate ID            0
      Job Role                4
      Application Date        0
      Screening Status        0
      Interview Score      1137
      Interview Status        0
      Offer Status            0
      Joining Status          0
      Recruitment Source      5
      Joining Date         2298
      dtype: int64
```

Screenshot: missing values in each column

1.6 Unique labels before cleaning

The unique values of every label column were printed. The same label appears in different forms: 'Passed' and 'passed'; 'Not Scheduled' and 'NOT SCHEDULED'; 'Not Offered' and 'Not Offered ' (with an extra space); 'Not Joined' and 'not joined'; and 'Indeed' and ' indeed '. Pandas treats each spelling as a different value, so the stage counts would be wrong without cleaning.

```
[12]: df["Job Role"].unique()  # Check unique values in job role column

[12]: array(['Sales Executive', 'Customer Support', 'Software Engineer',
       'Marketing Specialist', 'HR Executive', 'Project Manager',
       'Finance Analyst', 'Data Analyst', nan], dtype=object)

[13]: df["Screening Status"].unique()  # Check unique values in screening status colum

[13]: array(['Passed', 'Rejected', 'Pending', 'passed'], dtype=object)

[14]: df["Interview Status"].unique()  # Check unique values in interview status column

[14]: array(['Selected', 'Not Scheduled', 'Rejected', 'Pending',
       'NOT SCHEDULED', 'PENDING', 'REJECTED'], dtype=object)

[15]: df["Offer Status"].unique()  # Check unique values in offer status column

[15]: array(['Accepted', 'Not Offered', 'Pending', 'Declined', 'Not Offered '],
      dtype=object)

[16]: df["Joining Status"].unique()  # Check unique values in joining status column

[16]: array(['Joined', 'Not Joined', 'Withdrawn', 'Pending', 'not joined'],
      dtype=object)

[17]: df["Recruitment Source"].unique()  # Check unique values in recruitment source column

[17]: array(['Employee Referral', 'Naukri', 'Recruitment Agency',
       'Campus Recruitment', 'Indeed', 'LinkedIn', 'Company Website',
       ' indeed ', nan, ' recruitment agency ', ' employee referral ',
       ' company website ', ' campus recruitment '], dtype=object)
```

Screenshot: unique labels of the stage columns and Recruitment Source before cleaning

1.7 Summary: how every column was handled

Column	Problem found	How it was handled	Why
Job Role	Capital letters and spaces; 4 missing	str.strip().str.title(); missing values left as they are	Same role must be one group
Screening Status	'passed' and 'Passed'	Trimmed spaces and converted to title case; 3 labels remain	Same stage result must be counted together
Interview Status	Upper case labels (NOT SCHEDULED, PENDING, REJECTED)	Trimmed spaces and converted to title case; 4 labels remain	Same stage result must be counted together
Offer Status	'Not Offered ' with an extra space	Trimmed spaces and converted to title case; 4 labels remain	Same stage result must be counted together
Joining Status	'not joined' in lower case	Trimmed spaces and converted to title case; 4 labels remain	Same stage result must be counted together
Recruitment Source	Spaces and lower case (e.g. " indeed "); 5 missing	Trimmed spaces and converted to title case; 7 sources remain	Same source must be one group

Column	Problem found	How it was handled	Why
Interview Score	1,137 missing; maximum 11	Missing values filled with the median (6.5)	Median is not pulled by extreme scores
Application Date, Joining Date	Stored as text	Converted to datetime in Task 5	Needed to calculate hiring time
All rows	6 exact duplicate rows	Removed with drop_duplicates()	Same candidate must be counted once

1.8 Standardising the labels

For every label column the same two steps were used: str.strip() removes extra spaces and str.title() writes each word with a capital first letter. Then unique() was run again to confirm the result. The check shows that every column now has only its proper labels.

Job Role: the 8 roles stay the same. One small side effect is that 'HR Executive' becomes 'Hr Executive', because title case changes every letter after the first to lower case.

```
[19]: df["Job Role"] = df["Job Role"].str.strip().str.title()  # Correct inconsistent job role names

[20]: df["Job Role"].unique()  # Check unique values again

[20]: array(['Sales Executive', 'Customer Support', 'Software Engineer',
       'Marketing Specialist', 'Hr Executive', 'Project Manager',
       'Finance Analyst', 'Data Analyst', nan], dtype=object)
```

Screenshot: Job Role: cleaning the names

Screening Status and Interview Status: the 'passed' label is merged into 'Passed', and the three upper case interview labels are merged with their normal forms.

```
[22]: df["Screening Status"] = df["Screening Status"].str.strip().str.title()  # Correct inconsistent values

[23]: df["Screening Status"].unique()  # Check unique values again

[23]: array(['Passed', 'Rejected', 'Pending'], dtype=object)

[25]: df["Interview Status"] = df["Interview Status"].str.strip().str.title()  # Correct inconsistent values

[26]: df["Interview Status"].unique()  # Check unique values again

[26]: array(['Selected', 'Not Scheduled', 'Rejected', 'Pending'], dtype=object)
```

Screenshot: Screening Status and Interview Status after cleaning

Offer Status and Joining Status: 'Not Offered ' (with a space) is merged with 'Not Offered', and 'not joined' is merged with 'Not Joined'.

```
[28]: df["Offer Status"] = df["Offer Status"].str.strip().str.title()  # Correct inconsistent values

[29]: df["Offer Status"].unique()  # Check unique values again

[29]: array(['Accepted', 'Not Offered', 'Pending', 'Declined'], dtype=object)

[31]: df["Joining Status"] = df["Joining Status"].str.strip().str.title()  # Correct inconsistent values

[32]: df["Joining Status"].unique()  # Check unique values again

[32]: array(['Joined', 'Not Joined', 'Withdrawn', 'Pending'], dtype=object)
```

Screenshot: Offer Status and Joining Status after cleaning

Recruitment Source: labels such as ' indeed ' and ' employee referral ' are merged with their proper forms, so 7 sources remain. Title case writes 'LinkedIn' as 'Linkedin', which is only a spelling change.

```
[34]: df["Recruitment Source"] = df["Recruitment Source"].str.strip().str.title()  # Correct inconsistent values

[35]: df["Recruitment Source"].unique()  # Check unique values again

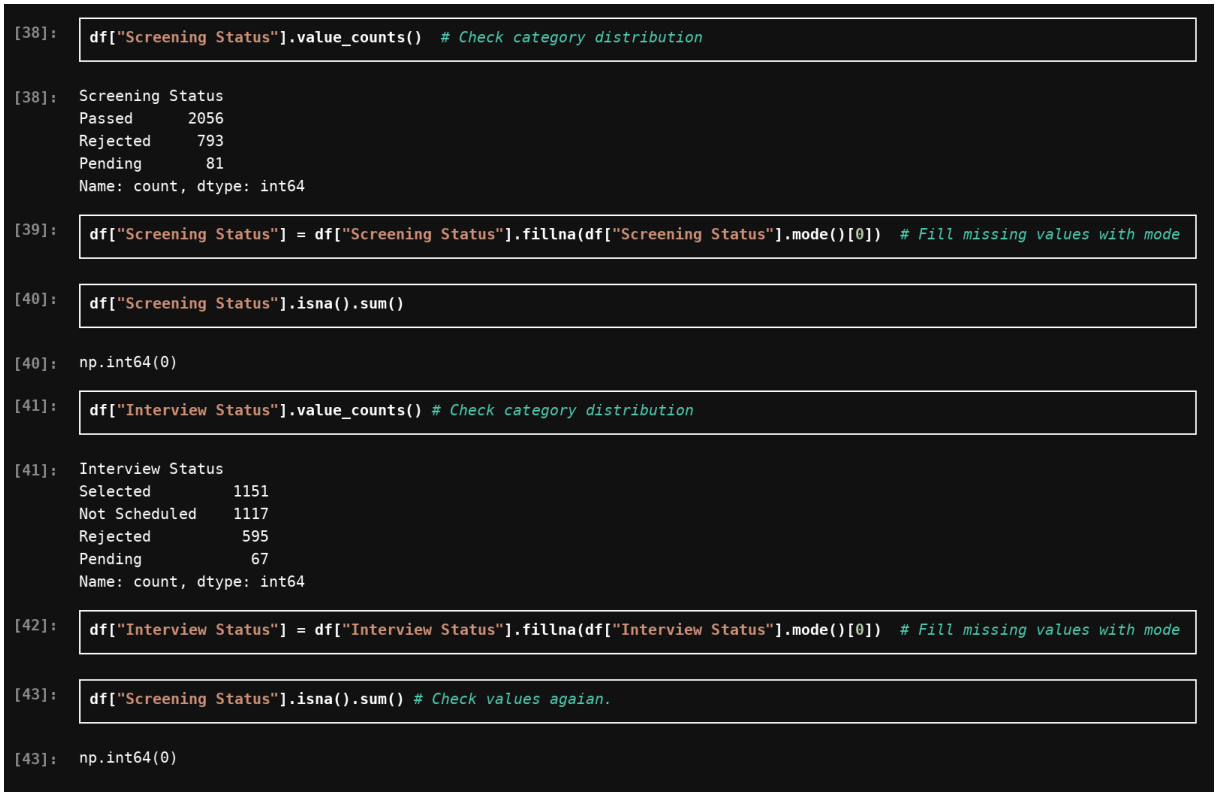
[35]: array(['Employee Referral', 'Naukri', 'Recruitment Agency',
             'Campus Recruitment', 'Indeed', 'Linkedin', 'Company Website', nan],
            dtype=object)
```

Screenshot: Recruitment Source after cleaning

1.9 Filling the missing values

The four stage columns (Screening, Interview, Offer and Joining Status) had no missing values, so filling them with the most common value (mode) or with 'Unknown' did not change them. The columns that really had missing values were handled as follows.

Screening Status and Interview Status: value_counts() shows the distribution and fillna() with the mode was applied. No value was missing, so nothing changed.



```
[38]: df["Screening Status"].value_counts()  # Check category distribution

[38]: Screening Status
      Passed      2056
      Rejected     793
      Pending       81
      Name: count, dtype: int64

[39]: df["Screening Status"] = df["Screening Status"].fillna(df["Screening Status"].mode()[0])  # Fill missing values with mode

[40]: df["Screening Status"].isna().sum()

[40]: np.int64(0)

[41]: df["Interview Status"].value_counts() # Check category distribution

[41]: Interview Status
      Selected         1151
      Not Scheduled    1117
      Rejected          595
      Pending            67
      Name: count, dtype: int64

[42]: df["Interview Status"] = df["Interview Status"].fillna(df["Interview Status"].mode()[0])  # Fill missing values with mode

[43]: df["Screening Status"].isna().sum() # Check values agaian.

[43]: np.int64(0)
```

Screenshot: Screening Status and Interview Status: distribution and filling

Offer Status and Joining Status: these were filled with 'Unknown' where a value was missing. The counts are the same before and after, which confirms that there were no missing values.

```
[44]: df["Offer Status"].value_counts()  # Check category distribution

[44]: Offer Status
      Not Offered    2006
      Accepted        708
      Declined        175
      Pending          41
      Name: count, dtype: int64

[45]: df["Offer Status"] = df["Offer Status"].fillna("Unknown")  # Fill missing values with mode

[46]: df["Offer Status"].isna().sum()  # Check missing values

[46]: np.int64(0)

[47]: df["Offer Status"].value_counts()  # Check category distribution

[47]: Offer Status
      Not Offered    2006
      Accepted        708
      Declined        175
      Pending          41
      Name: count, dtype: int64

[48]: df["Joining Status"] = df["Joining Status"].fillna("Unknown")  # Fill missing values with Unknown

[49]: df["Joining Status"].isna().sum()  # Check missing values

[49]: np.int64(0)
```

Screenshot: Offer Status and Joining Status: distribution and filling

Recruitment Source: 5 values are missing. The code fillna(mode()) did not fill them, so isna().sum() still shows 5. These 5 candidates are left out of the source comparison in Task 4, which is why the source counts add up to 2,919 and not 2,924.

```
[50]: df["Recruitment Source"].value_counts()  # Check category distribution

[50]: Recruitment Source
      Linkedin              711
      Naukri                658
      Employee Referral     439
      Indeed                360
      Company Website       329
      Campus Recruitment    248
      Recruitment Agency    180
      Name: count, dtype: int64

[51]: df["Recruitment Source"] = df["Recruitment Source"].fillna(df["Recruitment Source"].mode())  # Fill only missing values

[52]: df["Recruitment Source"].isna().sum()  # Check missing values

[52]: np.int64(5)

[53]: df["Recruitment Source"].value_counts()  # Check category distribution

[53]: Recruitment Source
      Linkedin              711
      Naukri                658
      Employee Referral     439
      Indeed                360
      Company Website       329
      Campus Recruitment    248
      Recruitment Agency    180
      Name: count, dtype: int64
```

Screenshot: Recruitment Source: distribution and filling

Interview Score: 1,137 scores are missing. The median (6.5) was used to fill them, because the median is not changed by a few very high or very low scores.

```
[54]: df["Interview Score"].describe()  # Check statistical summary

[54]: count    1793.000000
      mean        6.479922
      std         1.532807
      min         0.000000
      25%         5.400000
      50%         6.500000
      75%         7.600000
      max        11.000000
      Name: Interview Score, dtype: float64

[57]: df["Interview Score"].median()  # Check median score

[57]: 6.5

[58]: df["Interview Score"] = df["Interview Score"].fillna(df["Interview Score"].median()) # Fill not/null with median.
```

Screenshot: Interview Score: summary and filling with the median

1.10 Duplicate records

duplicated().sum() showed **6** exact duplicate rows. A duplicate is a row that is repeated exactly, which would count the same candidate twice. These rows were removed with drop_duplicates() and a second check shows 0 duplicates. The dataset now has **2,924 rows**.

```
[60]: df.duplicated().sum()  # Check duplicate records

[60]: np.int64(6)

[61]: df = df.drop_duplicates() # Removed Duplicates records.

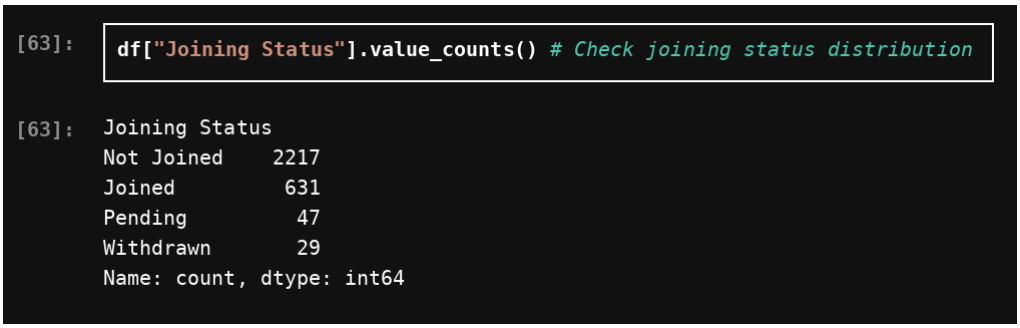
[62]: df.duplicated().sum()  # Check duplicate records

[62]: np.int64(0)
```

Screenshot: duplicate rows before and after removal

Part 3 - Task 2: Calculate the Number of Candidates at Each Recruitment Stage

After cleaning, value_counts() was used on each stage column to count the candidates. The screenshot shows the Joining Status counts; the counts of the other stages are printed in Task 3. The funnel has four stages after applying: the candidate passes the screening, is selected in the interview, accepts the offer and joins the company.



Screenshot: candidates by Joining Status

Stage	Result of the stage (number of candidates)	Reached next stage
Applied	All applicants: 2,924	-
Screening	Passed 2,053; Rejected 790; Pending 81	2,053
Interview	Selected 1,150; Not Scheduled 1,113; Rejected 595; Pending 66	1,150
Offer	Accepted 707; Not Offered 2,001; Declined 175; Pending 41	707
Joining	Joined 631; Not Joined 2,217; Pending 47; Withdrawn 29	631

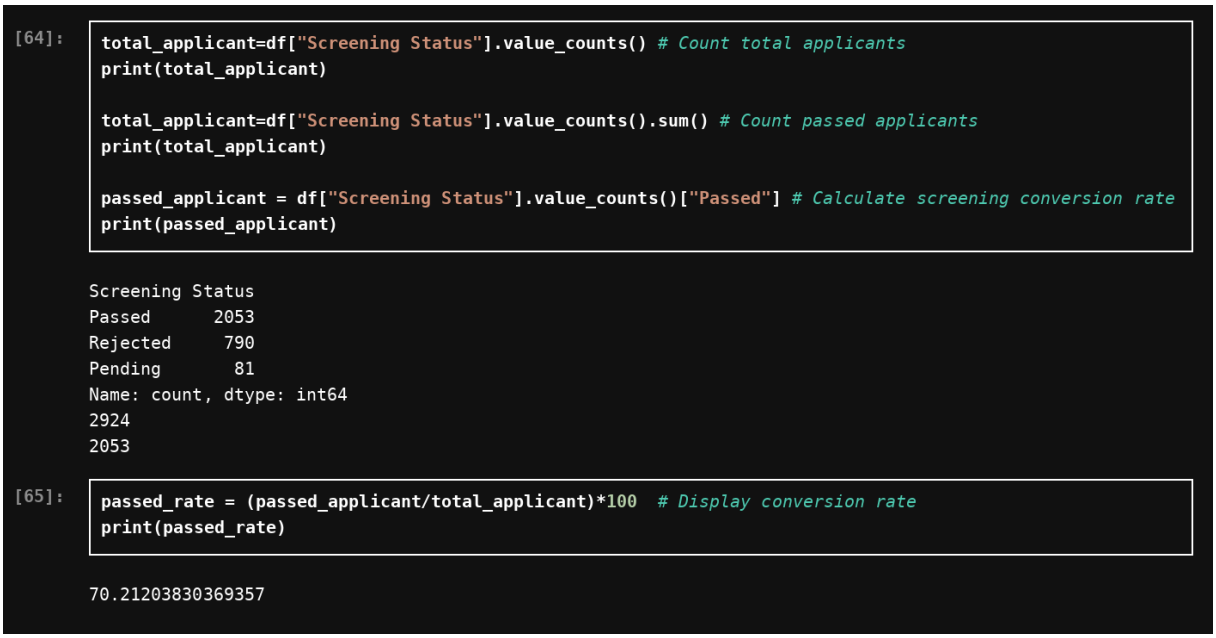
The number of candidates becomes smaller at every stage: **2,053** passed the screening, **1,150** were selected in the interview, **707** accepted the offer and **631** joined the company.

Part 3 - Task 3: Screening, Interview, Offer Acceptance and Joining Conversion Rates

A conversion rate shows what percentage of candidates reached a stage. In the notebook every rate is calculated against the total number of applicants (2,924): the count of the stage divided by the total, multiplied by 100. The code first prints the counts with value_counts(), then takes the count of the successful label (Passed, Selected, Accepted or Joined) and divides it by the total.

3.1 Screening conversion rate

2,053 of 2,924 candidates passed the screening, so the screening conversion rate is **70.21%**.



```
[64]: total_applicant=df["Screening Status"].value_counts() # Count total applicants
      print(total_applicant)

      total_applicant=df["Screening Status"].value_counts().sum() # Count passed applicants
      print(total_applicant)

      passed_applicant = df["Screening Status"].value_counts()["Passed"] # Calculate screening conversion rate
      print(passed_applicant)

Screening Status
Passed      2053
Rejected     790
Pending       81
Name: count, dtype: int64
2924
2053

[65]: passed_rate = (passed_applicant/total_applicant)*100  # Display conversion rate
      print(passed_rate)

70.21203830369357
```

Screenshot: screening conversion rate

3.2 Interview conversion rate

1,150 candidates were selected in the interview, so the rate is **39.33%**. Many candidates (1,113) were never scheduled for an interview, which keeps this rate low.

```
[66]: total_interview = df["Interview Status"].value_counts() # Count interview status
      print(total_interview)

      total_interview = df["Interview Status"].value_counts().sum() # Count total candidates
      print(total_interview)

      passed_interview = df["Interview Status"].value_counts()["Selected"] # Count selected candidates
      print(passed_interview)
```

```
Interview Status
Selected         1150
Not Scheduled    1113
Rejected          595
Pending            66
Name: count, dtype: int64
2924
1150
```

```
[67]: selected_rate = passed_interview/total_interview*100 # Calculate interview conversion rate
      print(selected_rate)
```

```
39.3296853625171
```

Screenshot: interview conversion rate

3.3 Offer acceptance rate

707 candidates accepted the offer, so the offer acceptance rate is **24.18%**.

```
[68]: offer_total = df["Offer Status"].value_counts() # Count offer status
      print(offer_total)

      offer_total = df["Offer Status"].value_counts().sum() # Count total offers
      print(offer_total)

      offer_accepted = df["Offer Status"].value_counts()["Accepted"] # Count accepted offers
      print(offer_accepted)
```

```
Offer Status
Not Offered    2001
Accepted        707
Declined        175
Pending          41
Name: count, dtype: int64
2924
707
```

```
[69]: offer_rate = offer_accepted/offer_total*100 # Calculate offer acceptance rate
      print(offer_rate)
```

```
24.17920656634747
```

Screenshot: offer acceptance rate

3.4 Joining conversion rate

631 candidates joined the company, so the joining conversion rate is **21.58%**.

```
[70]: joining_total = df["Joining Status"].value_counts() # Count joining status
      print(joining_total)

      joining_total = df["Joining Status"].value_counts().sum() # Count total candidates
      print(joining_total)

      joining_joined = df["Joining Status"].value_counts()["Joined"] # Count joined candidates
      print(joining_joined)

      Joining Status
      Not Joined    2217
      Joined         631
      Pending         47
      Withdrawn       29
      Name: count, dtype: int64
      2924
      631

[71]: joining_rate = joining_joined/joining_total*100 # Calculate joining conversion rate
      print(joining_rate)

      21.58002735978112
```

Screenshot: joining conversion rate

3.5 Combined result

Rate	Candidates	Calculation	Result
Screening conversion	2,053	2,053 / 2,924 x 100	70.21%
Interview conversion	1,150	1,150 / 2,924 x 100	39.33%
Offer acceptance	707	707 / 2,924 x 100	24.18%
Joining conversion	631	631 / 2,924 x 100	21.58%

Note: because all four rates use the same total (2,924), they show what share of all applicants reached each stage. If every stage is compared with the stage before it, the rates are: screening to interview 56.02% (1,150 of 2,053), interview to offer acceptance 61.48% (707 of 1,150) and offer acceptance to joining 89.25% (631 of 707).

Part 3 - Task 4: Compare Recruitment Sources by Candidate Conversion Rate

For every recruitment source the number of candidates was counted, then the number of candidates who passed the screening was counted, and the conversion rate was calculated as passed candidates divided by total candidates x 100.

4.1 Candidates from each source

```
[73]: Recruitment_total = df["Recruitment Source"].value_counts() # Count candidates from each recruitment source
      print(Recruitment_total)

Recruitment Source
Linkedin              708
Naukri                657
Employee Referral     438
Indeed                360
Company Website       328
Campus Recruitment    248
Recruitment Agency    180
Name: count, dtype: int64
```

Screenshot: number of candidates from each recruitment source

4.2 Candidates who passed the screening

```
[74]: screening_total = df["Recruitment Source"].value_counts() # Count total candidates by recruitment source
      print(screening_total)

      print("PASSED CANDIDATES :")

      screening_passed = df[df["Screening Status"] == "Passed"]["Recruitment Source"].value_counts()
      print(screening_passed) # Count passed candidates for each recruitment source

Recruitment Source
Linkedin              708
Naukri                657
Employee Referral     438
Indeed                360
Company Website       328
Campus Recruitment    248
Recruitment Agency    180
Name: count, dtype: int64
PASSED CANDIDATES :
Recruitment Source
Linkedin              503
Naukri                442
Employee Referral     368
Indeed                237
Company Website       216
Campus Recruitment    154
Recruitment Agency    129
Name: count, dtype: int64
```

Screenshot: candidates who passed the screening from each source

4.3 Conversation rate of each source

```
[75]: conversation_rate  = screening_passed/screening_total*100 # Calculate conversion rate for each recruitment source
      print(conversation_rate)

      Recruitment Source
      Linkedin              71.045198
      Naukri                67.275495
      Employee Referral     84.018265
      Indeed                65.833333
      Company Website       65.853659
      Campus Recruitment    62.096774
      Recruitment Agency    71.666667
      Name: count, dtype: float64
```

Screenshot: conversion rate of each recruitment source

Recruitment source	Candidates	Passed screening	Conversion rate
Employee Referral	438	368	84.02%
Recruitment Agency	180	129	71.67%
Linkedin	708	503	71.05%
Naukri	657	442	67.28%
Company Website	328	216	65.85%
Indeed	360	237	65.83%
Campus Recruitment	248	154	62.10%

Result: Employee Referral has the highest conversion rate (84.02%), which means candidates who are recommended by employees are the most suitable. Recruitment Agency (71.67%) and LinkedIn (71.05%) come next. Campus Recruitment has the lowest rate (62.10%). LinkedIn and Naukri bring the largest number of candidates (708 and 657), but their rates are close to the middle.

Part 3 - Task 5: Average Hiring Time

Application Date and Joining Date were stored as text, so both were converted to datetime with pd.to_datetime(). The hiring time is the difference between the joining date and the application date in days, saved in a new column called Hiring Time. The average was then taken only for the candidates whose Joining Status is 'Joined'.

```
[76]: df.dtypes # Check data types of all columns

[76]: Candidate ID          object
      Job Role              object
      Application Date      object
      Screening Status      object
      Interview Score      float64
      Interview Status      object
      Offer Status          object
      Joining Status        object
      Recruitment Source    object
      Joining Date          object
      dtype: object

[77]: df["Application Date"] = pd.to_datetime(df["Application Date"]) # Convert application date to datetime
      df["Joining Date"] = pd.to_datetime(df["Joining Date"]) # Convert joining date to datetime

      df["Hiring Time"] = (df["Joining Date"] - df["Application Date"]).dt.days  # Calculate hiring time in days

      df[df["Joining Status"] == "Joined"]["Hiring Time"].mean()  ## Calculate average hiring time for joined candidates

[77]: np.float64(34.74484944532488)

[81]: df.columns

[81]: Index(['Candidate ID', 'Job Role', 'Application Date', 'Screening Status',
             'Interview Score', 'Interview Status', 'Offer Status', 'Joining Status',
             'Recruitment Source', 'Joining Date', 'Hiring Time'],
            dtype='object')
```

Screenshot: converting the dates and calculating the hiring time

The average hiring time is **34.74 days**, so a joined candidate needs about 35 days (5 weeks) from applying to joining the company.

Part 3 - Visualization Tasks

Task 6: Bar Chart - Candidates at Each Recruitment Stage

A bar chart shows how many candidates are left at each stage of the funnel.

[84]:

```
plt.figure(figsize=(11, 6), facecolor="#FFFFFF")
plt.gca().set_facecolor("#F8FAFC")

colors = ["#1F4E79", "#3B6A96", "#5B8DB8", "#82A9CC"]

bars = plt.bar(
    stages, counts,
    color=colors,
    edgecolor="#FFFFFF",
    linewidth=1,
    width=0.6
)

plt.ylim(0, max(counts) + 250)

plt.title("Candidates at Each Recruitment Stage",
          fontsize=15, fontweight="bold", color="#1E293B", pad=18)

plt.xlabel("Recruitment Stage", fontsize=11, color="#334155")
plt.ylabel("Number of Candidates", fontsize=11, color="#334155")

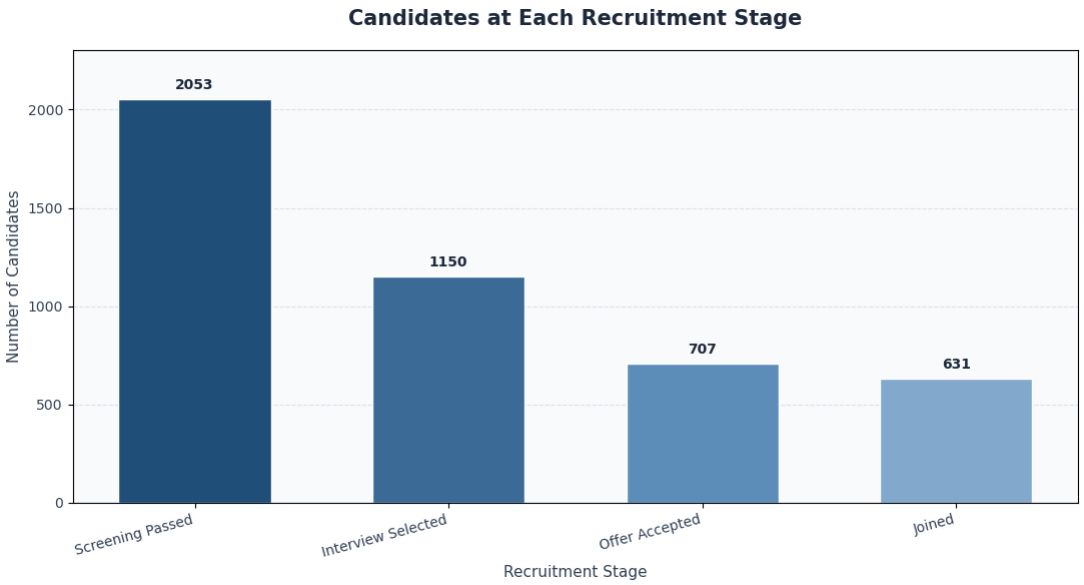
plt.xticks(rotation=15, ha="right", color="#334155")
plt.yticks(color="#334155")

plt.bar_label(bars, padding=5, fontsize=10,
              fontweight="bold", color="#1E293B")

plt.grid(axis="y", linestyle="--", color="#CBD5E1", alpha=0.6)
plt.gca().set_axisbelow(True)

plt.tight_layout()
plt.show()
```

Screenshot: code for the bar chart of recruitment stages



Screenshot: bar chart of candidates at each recruitment stage

What the chart shows: 2,053 candidates passed the screening, 1,150 were selected in the interview, 707 accepted the offer and 631 joined. The bars become shorter at every stage, and the largest fall is between screening and interview (903 candidates). The fall between offer acceptance and joining is the smallest (76 candidates).

Task 7: Bar Chart - Recruitment Source Conversion Rates

This bar chart compares the conversion rate of all recruitment sources.

[85]:

```
colors = ["#264653", "#2A9D8F", "#6A9FB5", "#A8C5D1"]

plt.figure(figsize=(10, 6), facecolor="#F5F7FA")
plt.gca().set_facecolor("#FFFFFF")

bars = plt.bar(name, values, color=colors[:len(name)],
               edgecolor="#DCE3EA", linewidth=1)

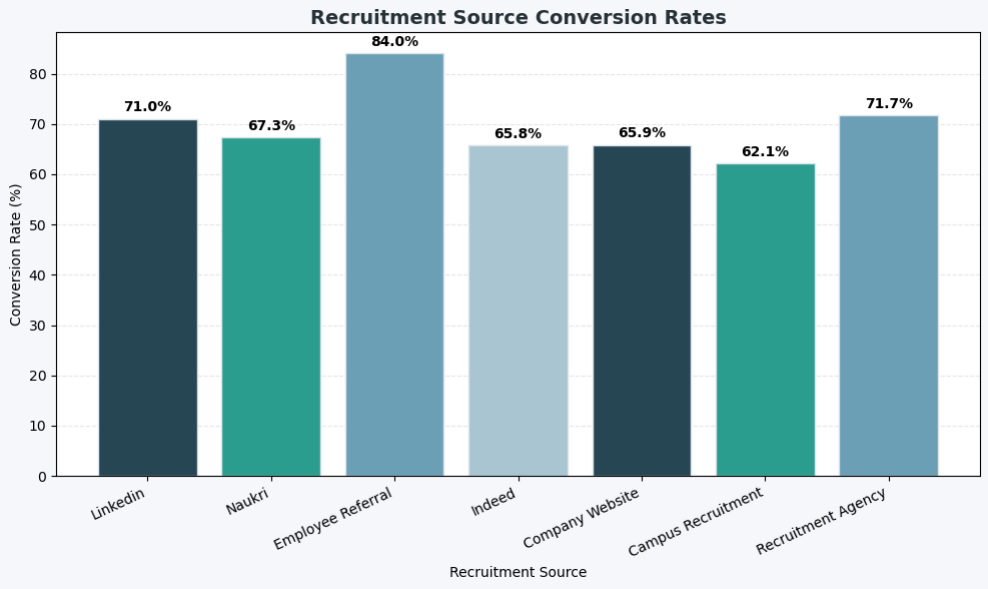
plt.title("Recruitment Source Conversion Rates",
          fontsize=14, fontweight="bold", color="#263238")

plt.xlabel("Recruitment Source")
plt.ylabel("Conversion Rate (%)")
plt.xticks(rotation=25, ha="right")

plt.grid(axis="y", linestyle="--", alpha=0.3)
plt.gca().set_axisbelow(True)

plt.bar_label(bars, fmt="%.1f%%", padding=3, fontweight="bold")
plt.tight_layout()
plt.show()
```

Screenshot: code for the source conversion rate chart



Screenshot: bar chart of recruitment source conversion rates

What the chart shows: Employee Referral is clearly the highest (84.0%). Recruitment Agency (71.7%) and LinkedIn (71.0%) follow, then Naukri (67.3%), Company Website (65.9%) and Indeed (65.8%). Campus Recruitment is the lowest (62.1%). The difference between the best and the lowest source is about 22 percentage points.

Task 8: Box Plot - Recruitment Turnaround Time by Job Role

A box plot shows how the hiring time (in days) is spread for each job role. The box covers the middle 50% of the candidates, the orange line is the median and the dots are unusually long cases (outliers).

[87]:

```
import matplotlib.pyplot as plt

fig, ax = plt.subplots(figsize=(12, 6), facecolor="#F5F7FA")
ax.set_facecolor("#FFFFFF")

box = ax.boxplot(
    [group["Hiring Time"].dropna().values
     for _, group in df.groupby("Job Role")],
    patch_artist=True,
    labels=df.groupby("Job Role").groups.keys()
)

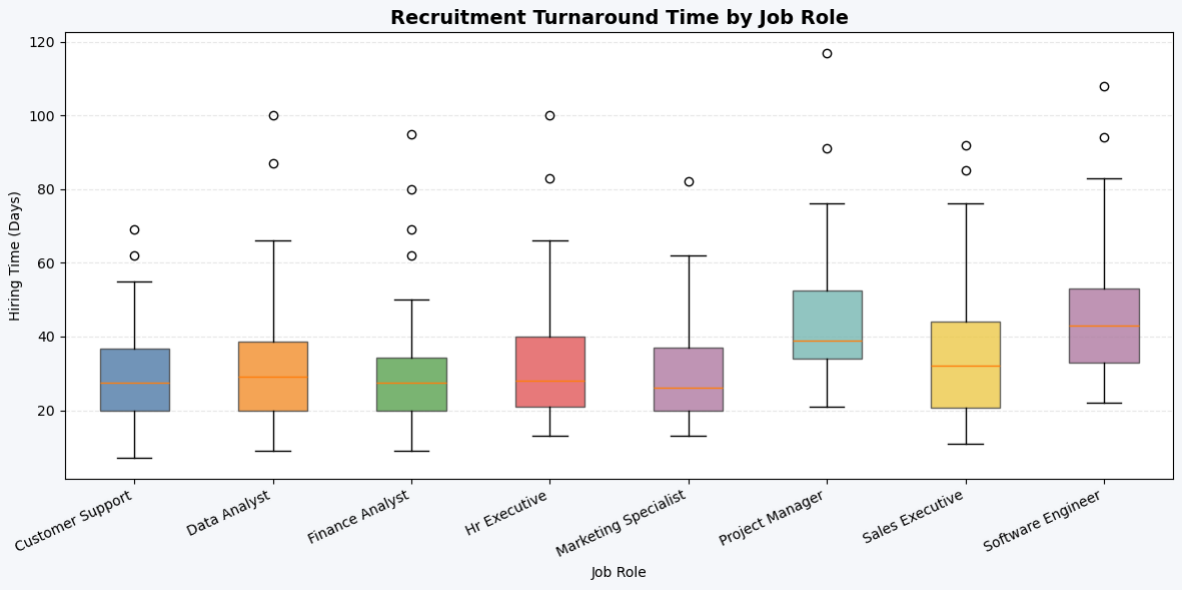
colors = ["#4E79A7", "#F28E2B", "#59A14F", "#E15759",
          "#B07AA1", "#76B7B2", "#EDC948", "#AF7AA1"]

for patch, color in zip(box["boxes"], colors):
    patch.set_facecolor(color)
    patch.set_alpha(0.8)
    patch.set_edgecolor("#444444")

plt.title("Recruitment Turnaround Time by Job Role",
          fontsize=14, fontweight="bold")
plt.xlabel("Job Role")
plt.ylabel("Hiring Time (Days)")
plt.xticks(rotation=25, ha="right")
plt.grid(axis="y", linestyle="--", alpha=0.3)
ax.set_axisbelow(True)

plt.tight_layout()
plt.show()
```

Screenshot: code for the box plot of hiring time by job role



Screenshot: box plot of recruitment turnaround time by job role

What the chart shows: Software Engineer and Project Manager need the longest time, with median hiring times of about 43 and 39 days. The other roles are lower; Marketing Specialist, Customer Support, Finance Analyst, HR Executive and Data Analyst have medians of about 26 to 29 days, and Sales Executive is about 32 days. Software Engineer and Project Manager also have the longest cases (more than 100 days), and Software Engineer, Project Manager and Sales Executive have the widest

boxes, which means their hiring time varies a lot. The values are read from the chart, so they are approximate.

Part 3 - Interpretation

Task 9: Where Most Candidates are Lost in the Recruitment Process

The number of candidates at each stage was compared to see where the funnel loses the most candidates.

Step in the funnel	Candidates before	Candidates lost	Share lost
Screening passed to interview selected	2,053	903	43.98%
Interview selected to offer accepted	1,150	443	38.52%
Offer accepted to joined	707	76	10.75%

Other numbers from the data: 790 candidates were rejected in the screening and 175 candidates declined their offer.

Conclusion: the biggest loss is between screening and interview, where **903** candidates are lost (43.98% of those who passed the screening). Many of them were never scheduled for an interview (1,113 candidates have the status Not Scheduled). The company should therefore focus most on this part of the process. The loss after the offer is small in number (76) but still important, because these candidates have already been selected, and 175 more declined the offer.

Task 10: Ways to Improve the Hiring Process

1. **Screen correctly:** call only suitable candidates, so that the interview time is not wasted and fewer candidates are lost between screening and interview.
2. **Schedule interviews quickly:** many screened candidates never get an interview, so the interview dates should be fixed soon after the screening.
3. **Competitive salary:** offer a salary that matches the skill and experience of the candidate, so that fewer candidates decline the offer (175 declined).
4. **Friendly working environment:** provide a positive workplace, because candidates compare it with other companies before they accept.
5. **Fast hiring process:** do not take too long to finish the hiring, so that candidates do not lose interest. The average is about 35 days, and Software Engineer and Project Manager roles take longer.
6. **Describe the job clearly:** tell the candidate exactly what the job involves when making the offer.
7. **Use employee referrals more:** referred candidates have the highest screening pass rate (84.02%).
8. **Stay in touch after the offer:** keep contact with candidates until the joining date, because 76 candidates who accepted did not join.

Limitations

- Interview Score was missing for 1,137 candidates (about 39%) and was filled with the median (6.5). Most of these candidates were never interviewed, so the filled scores are not real scores.
- The 5 missing Recruitment Source values and the 4 missing Job Role values were not filled, so those candidates are left out of the source and job role comparisons.
- Joining Date is empty for 2,298 candidates, mostly those who never joined, so the hiring time can be calculated only for candidates who joined.
- All conversion rates in Task 3 are calculated against all 2,924 applicants, not against the previous stage.

- The maximum Interview Score is 11, which may be a data entry error; it was not changed because the scoring scale is not given.
- The analysis is descriptive. It shows where candidates are lost but does not prove why they are lost.

Part 3 Conclusion

In Part 3 the recruitment data was cleaned: the stage labels and recruitment source names were standardised, 6 duplicate rows were removed and the missing Interview Score values were filled with the median. The final dataset has 2,924 candidates.

Out of 2,924 applicants, **2,053** (70.21%) passed the screening, **1,150** (39.33%) were selected in the interview, **707** (24.18%) accepted the offer and **631** (21.58%) joined the company. Employee Referral was the best recruitment source (84.02% screening pass rate) and Campus Recruitment the lowest (62.10%). The average hiring time was 34.74 days, and Software Engineer and Project Manager roles took the longest.

The most candidates were lost between screening and interview (903 candidates). To improve the hiring process, the company should screen candidates more carefully, schedule interviews faster, offer competitive salaries, shorten the hiring time, describe the job clearly and use employee referrals more. In future, the same funnel can be studied by job role or by month to find more detailed problems.

PART 4: Salary Distribution and Compensation Analysis

About Part 4

Compensation analysis studies how much employees are paid and which factors are linked with pay. Part 4 uses an employee salary file to check the salary and experience data, calculate salary statistics, compare salaries across departments, designations, experience groups and education levels, and find unusually high or low salaries.

Part 4 dataset: Task 4.csv - 2,750 rows and 8 columns.

Column	Meaning
Employee ID	Unique identification of each employee
Department	Department in which the employee works
Designation	Job position of the employee
Experience	Total work experience in years
Education Level	Highest education of the employee (High School, Diploma, Bachelor's, Master's, MBA, PhD)
Salary	Salary received by the employee
Performance Rating	Performance rating of the employee from 1 to 5 (not used in this analysis)
Location	City in which the employee works

Objectives of Part 4

- To clean the salary and experience fields and handle missing records.
- To calculate the mean, median and standard deviation of salaries.
- To calculate department-wise and designation-wise average salaries.
- To compare salary levels across experience groups and education levels.
- To identify unusually high or low salaries with a statistical method.
- To visualise salary distributions with box plots, a scatter plot and grouped bar charts.

Quick summary of results (Part 4)

Item	Result
Records in raw file	2,750 rows x 8 columns (22,000 values)
Missing values found	55 (Experience 33, Salary 22), about 0.25% of all values
Department names	11 different spellings cleaned into 8 departments
Duplicate employees	None (2,750 unique Employee IDs in 2,750 rows)
Salary: mean / median / std. deviation	68,370.24 / 64,500 / 23,978.82
Highest / lowest paying department	Technology (93,735.88) / Administration (46,757.56)
Highest / lowest paying designation	Engineering Manager (127,145.57) / Business Development Executive (50,601.45)
Average salary by experience group	Low 60,842.14, Mid 68,445.79, High 76,881.14
Average salary by education	PhD 75,347.83 (highest) to High School 65,412.62 (lowest)
Unusual salaries (IQR method)	59 salaries above the upper limit of 128,187.50; none below the lower limit

Part 4 - Task 1: Load the Dataset, Check Its Quality and Clean Salary and Experience

First, the libraries were imported and the CSV file was loaded into a Pandas DataFrame called df. The data was then examined for its size, data types, missing values and inconsistent labels, and the problems were fixed before the analysis.

1.1 Import libraries and load the data

Pandas is used for tables and calculations, NumPy for numbers, and Matplotlib for charts.

[1]:
```
# Import libraries :
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

[2]:
```
# Load dataSet :
df = pd.read_csv(r"C:\Users\vssag\Desktop\Task 4.csv")
```

Screenshot: importing libraries and loading the CSV file

1.2 First look at the data

head() shows the first five rows and tail() shows the last five rows. Each row is one employee.

[3]:
```
df.head() # It shows top five rows
```

[3]:

	Employee ID	Department	Designation	Experience	Education Level	Salary	Performance Rating	Location
0	EMP00001	Human Resources	HR Executive	0.9	MBA	57500.0	4	Gurugram
1	EMP00002	Technology	IT Support Specialist	2.3	Diploma	79000.0	4	Delhi
2	EMP00003	Human Resources	HR Executive	0.1	Bachelor's	52000.0	4	Bengaluru
3	EMP00004	Administration	Operations Executive	2.9	Bachelor's	42500.0	1	Jaipur
4	EMP00005	Marketing	Marketing Executive	2.7	Master's	46000.0	3	Delhi

[4]:
```
df.tail() # It shows last five rows
```

[4]:

	Employee ID	Department	Designation	Experience	Education Level	Salary	Performance Rating	Location
2745	EMP02746	Operations	Operations Executive	4.3	Bachelor's	74000.0	3	Pune
2746	EMP02747	Finance	Finance & Accounting Specialist	1.7	Bachelor's	57500.0	3	Pune
2747	EMP02748	Technology	Engineering Manager	5.1	Diploma	132000.0	4	Noida
2748	EMP02749	Operations	Operations Executive	16.6	Bachelor's	76500.0	5	Noida
2749	EMP02750	Customer Support	IT Support Specialist	13.3	Master's	70500.0	2	Gurugram

Screenshot: df.head() and df.tail()

1.3 Shape and column names

The dataset has **2,750 rows and 8 columns**.

```
[5]: df.shape # It will give dataset size(rows,column)

[5]: (2750, 8)

[6]: print(df.columns.tolist()) # It shows All column names in a list.

['Employee ID', 'Department', 'Designation', 'Experience', 'Education Level', 'Salary',
    'Performance Rating', 'Location']
```

Screenshot: shape and column names

1.4 Dataset information, statistical summary and data types

The info() output shows that Experience has 2,717 filled values and Salary has 2,728, so both have missing values. Experience and Salary are decimal numbers (float64), Performance Rating is a whole number (int64) and the other columns are text (object).

```
[7]: df.info() # It will give Dataset structure.

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2750 entries, 0 to 2749
Data columns (total 8 columns):
 #   Column              Non-Null Count  Dtype
---  ------              --------------  -----
 0   Employee ID         2750 non-null   object
 1   Department          2750 non-null   object
 2   Designation         2750 non-null   object
 3   Experience          2717 non-null   float64
 4   Education Level     2750 non-null   object
 5   Salary              2728 non-null   float64
 6   Performance Rating  2750 non-null   int64
 7   Location            2750 non-null   object
dtypes: float64(2), int64(1), object(5)
memory usage: 172.0+ KB
```

Screenshot: df.info()

The describe() table shows that salaries range from 18,000 to 209,544 with an average of about 68,401, and experience ranges from 0 to 24.9 years with an average of about 6.17 years. Performance Rating goes from 1 to 5. All values are possible in real life, and there are no negative numbers.

[8]:
```
df.describe() # It shows statistical summary of the dataset
```

[8]:

	Experience	Salary	Performance Rating
count	2717.000000	2728.000000	2750.000000
mean	6.172175	68401.451246	3.170909
std	5.113335	24072.822441	0.916220
min	0.000000	18000.000000	1.000000
25%	2.300000	51000.000000	3.000000
50%	4.800000	64500.000000	3.000000
75%	8.400000	82000.000000	4.000000
max	24.900000	209544.000000	5.000000

[9]:
```
df.dtypes # it will shows Data types of columns.
```

[9]:
```
Employee ID            object
Department             object
Designation            object
Experience            float64
Education Level        object
Salary                float64
Performance Rating      int64
Location               object
dtype: object
```

Screenshot: df.describe() and df.dtypes

1.5 Missing values and unique values

isnull().sum() shows **55** missing values: Experience (33) and Salary (22). nunique() shows that Employee ID has 2,750 unique values, equal to the number of rows, so no employee is repeated. It also shows 11 different Department names, although a company like this has only 8 departments, which points to inconsistent spelling.

```
[10]: df.isnull().any() # Check any missing value in dataset by each column.

[10]: Employee ID          False
      Department           False
      Designation          False
      Experience            True
      Education Level      False
      Salary                True
      Performance Rating   False
      Location             False
      dtype: bool

[11]: df.isnull().sum() # It shows all misiing not/nll in each column.

[11]: Employee ID           0
      Department            0
      Designation           0
      Experience           33
      Education Level       0
      Salary               22
      Performance Rating    0
      Location              0
      dtype: int64

[12]: df.nunique() # It will give The count of nunique values.

[12]: Employee ID          2750
      Department             11
      Designation            15
      Experience            231
      Education Level         6
      Salary                255
      Performance Rating      5
      Location               14
      dtype: int64
```

Screenshot: missing values and unique values per column

1.6 Department: inconsistent labels

The unique department names show the same department written in different ways: 'Technology' and 'technology', 'Customer Support' and 'Cust. Support', and 'Human Resources' and 'HR'. This would split one department into several groups.

```
[13]: print(df["Department"].unique().tolist())# It shows unique value in department column.

['Human Resources', 'Technology', 'Administration', 'Marketing', 'Customer Support',
    'Operations', 'Sales', 'Finance', 'technology', 'Cust. Support', 'HR']
```

Screenshot: unique department names before cleaning

The names were cleaned in two steps: str.strip().str.title() removes extra spaces and writes each word with a capital first letter, and replace() changes 'Cust. Support' to 'Customer Support' and 'Hr' to 'Human Resources'. After cleaning, 8 departments remain.

```
[14]: df["Department"] = df["Department"].str.strip().str.title().replace({
          "Cust. Support": "Customer Support",
          "Hr": "Human Resources"
      })

[15]: print(df["Department"].unique().tolist()) # Check again for confirmation.

['Human Resources', 'Technology', 'Administration', 'Marketing', 'Customer Support',
    'Operations', 'Sales', 'Finance']
```

Screenshot: department cleaning and the result

1.7 Designation

The 15 designation names were checked in the same way. All of them are written consistently, so no change was needed.

```
[16]: print(df["Designation"].unique().tolist()) # It shows unique value in Designation column.

['HR Executive', 'IT Support Specialist', 'Operations Executive', 'Marketing Executive',
    'Software Engineer', 'Engineering Manager', 'Sales Executive', 'Data Analyst', 'Finance &
    Accounting Specialist', 'HR Manager', 'Sales Manager', 'Account Manager', 'QA Engineer',
    'Business Development Executive', 'DevOps Engineer']
```

Screenshot: unique designation names

1.8 Summary: how every column was handled

Column	Problem found	How it was handled	Why
Department	11 spellings (e.g. technology, Cust. Support, HR)	Trimmed spaces, title case and replace(); now 8 departments	Same department must be counted as one group
Designation	15 designations checked	No change needed	Names were already consistent
Salary	22 missing; range 18,000 to 209,544	Missing values filled with the median (64,500)	The median is not pulled by very high salaries
Experience	33 missing; range 0 to 24.9	Missing values filled with the median (4.8)	The median is not pulled by a few very long careers
Employee ID	Checked for repeats	No duplicates found	Each employee is counted once

1.9 Salary: checking the values

The frequency of each salary, the highest salary (209,544), the lowest salary (18,000) and the sorted values were checked to find impossible or unusual numbers. All salaries are positive and in a realistic range. The sorted list ends with NaN, which are the missing values.

```
[17]: # Clean Salary column.
      df["Salary"].value_counts() # Check the frequency of each salary value

[17]: Salary
      62000.0     43
      55000.0     36
      46500.0     35
      53000.0     35
      57000.0     32
                  ..
      146000.0     1
      131000.0     1
      82846.0      1
      137000.0     1
      132000.0     1
      Name: count, Length: 255, dtype: int64

[18]: df["Salary"].max() # Find the highest salary.

[18]: 209544.0

[19]: df["Salary"].min() # Find the minimum salary

[19]: 18000.0

[20]: df["Salary"].sort_values() # Sort Salary values in ascending order to identify potential outliers

[20]: 2556    18000.0
      1528    18000.0
      1366    18000.0
      813     18000.0
      2575    18000.0
              ...
      2435       NaN
      2550       NaN
      2637       NaN
      2677       NaN
      2693       NaN
      Name: Salary, Length: 2750, dtype: float64
```

Screenshot: checking the Salary column

1.10 Experience: checking the values

The same checks were done for Experience. The lowest value is 0 years and the highest is 24.9 years, which are possible values.

```
[21]: # Clean the Experience column.
      df["Experience"].value_counts() # Check the frequency of Experience each Experience value

[21]: Experience
      0.5     45
      1.8     41
      4.8     39
      4.4     38
      4.2     37
              ..
      23.1     1
      21.5     1
      19.3     1
      22.5     1
      23.6     1
      Name: count, Length: 231, dtype: int64

[22]: df["Experience"].min() # Find the lowest Experience

[22]: 0.0

[23]: df["Experience"].max() #Find the highest Experience

[23]: 24.9

[24]: df["Experience"].sort_values() # Sort Experience values in ascending order to identify potential outliers

[24]: 428     0.0
      1747    0.0
      2643    0.0
      2364    0.0
      1366    0.0
             ...
      2489    NaN
      2506    NaN
      2550    NaN
      2560    NaN
      2668    NaN
      Name: Experience, Length: 2750, dtype: float64
```

Screenshot: checking the Experience column

1.11 Salary: missing values

22 salaries are missing. The summary shows a mean of 68,401 and a median of 64,500, and the histogram shows that most salaries are between about 40,000 and 90,000 with a long tail of higher salaries. Because the data is skewed to the right, the median was chosen to fill the missing values.

```
[25]:  # Handle missing records.
       df["Salary"].isna().sum() # count missing values in salary column

[25]:  np.int64(22)

[26]:  df["Salary"].describe() # It give stastical summary of salary column

[26]:  count      2728.000000
       mean      68401.451246
       std       24072.822441
       min       18000.000000
       25%       51000.000000
       50%       64500.000000
       75%       82000.000000
       max      209544.000000
       Name: Salary, dtype: float64

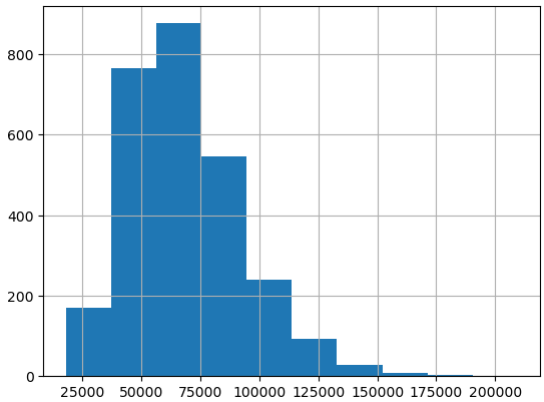
[27]:  df["Salary"].median() # It shows middle value of salary column.

[27]:  64500.0
```

Screenshot: Salary: missing values, summary and median

```
[28]:  df["Salary"].hist() #  Check the distribution of Salary
```

Screenshot: code for the Salary histogram



Screenshot: histogram of Salary

```
[29]:  df["Salary"] = df["Salary"].fillna(df["Salary"].median()) # Replcae missing value with middle value.

[30]:  df["Salary"].isna().sum() # Check again for confirmation

[30]:  np.int64(0)
```

Screenshot: Salary: filling with the median and checking

1.12 Experience: missing values

33 experience values are missing. The histogram shows that most employees have less than 10 years of experience and only a few have more than 15 years. The data is again skewed to the right, so the median (4.8 years) was used to fill the missing values.

```
[31]: df["Experience"].isna().sum() # Check not/null value in Experience column

[31]: np.int64(33)

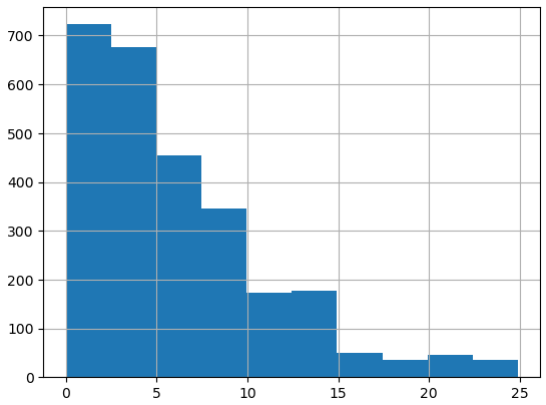
[32]: df["Experience"].describe() # provide Stastical summary of Experience.

[32]: count    2717.000000
      mean        6.172175
      std         5.113335
      min         0.000000
      25%         2.300000
      50%         4.800000
      75%         8.400000
      max        24.900000
      Name: Experience, dtype: float64
```

Screenshot: Experience: missing values and summary

```
[33]: df["Experience"].hist() #  Check the distribution of Experience.
```

Screenshot: code for the Experience histogram



Screenshot: histogram of Experience

```
[34]: df["Experience"] = df["Experience"].fillna(df["Experience"].median()) # Replcae missing value with middle value.

[35]: df["Experience"].isna().sum() # Check again for confirmation.

[35]: np.int64(0)
```

Screenshot: Experience: filling with the median and checking

After filling, both columns show 0 missing values.

Part 4 - Task 2: Mean, Median and Standard Deviation of Salaries

The mean is the average salary, the median is the middle salary and the standard deviation shows how much salaries differ from the mean. All of them come from one command, describe().

```
[36]: df["Salary"].describe() # It will give statistical summary of salary column including mean median and standard deviation.

[36]: count     2750.000000
      mean     68370.239636
      std      23978.822489
      min      18000.000000
      25%      51000.000000
      50%      64500.000000
      75%      81875.000000
      max     209544.000000
      Name: Salary, dtype: float64
```

Screenshot: statistical summary of Salary after cleaning

Measure	Value	Meaning
Mean	68,370.24	Average salary
Median	64,500.00	Middle salary
Standard deviation	23,978.82	Spread of salaries around the mean
Minimum / maximum	18,000 / 209,544	Lowest and highest salary
25% / 75%	51,000 / 81,875	Lower and upper quartile

The mean (68,370) is higher than the median (64,500), which means a few very high salaries pull the average up. The standard deviation is large, so salaries differ a lot between employees.

Part 4 - Task 3: Department-wise and Designation-wise Average Salaries

3.1 Average salary by department

groupby("Department")["Salary"].mean() gives the average salary of each department.

```
[37]: avg = df.groupby("Department")["Salary"].mean() # department wise salary average.
      print(avg)

      Department
      Administration      46757.559585
      Customer Support    49741.042345
      Finance             73884.395833
      Human Resources     63559.501718
      Marketing           62268.518519
      Operations          62332.475936
      Sales               60818.783069
      Technology          93735.875193
      Name: Salary, dtype: float64
```

Screenshot: average salary of each department

Department	Average salary
Technology	93,735.88
Finance	73,884.40
Human Resources	63,559.50
Operations	62,332.48
Marketing	62,268.52
Sales	60,818.78
Customer Support	49,741.04
Administration	46,757.56

Technology pays the most and is far above all other departments. Administration and Customer Support pay the least.

3.2 Average salary by designation

```
[38]: avg = df.groupby("Designation")["Salary"].mean() # Designatio wise salary average.
      print(avg)

Designation
Account Manager                     63136.363636
Business Development Executive      50601.449275
Data Analyst                        91757.664634
DevOps Engineer                    104536.231884
Engineering Manager                127145.569620
Finance & Accounting Specialist     70672.110465
HR Executive                        59912.977143
HR Manager                          83585.491525
IT Support Specialist               55356.435208
Marketing Executive                 62268.518519
Operations Executive                57040.518453
QA Engineer                         81373.563218
Sales Executive                     53438.311688
Sales Manager                       82141.025641
Software Engineer                   95282.612245
Name: Salary, dtype: float64
```

Screenshot: average salary of each designation

Designation	Average salary
Engineering Manager	127,145.57
DevOps Engineer	104,536.23
Software Engineer	95,282.61
Data Analyst	91,757.66
HR Manager	83,585.49
Sales Manager	82,141.03
QA Engineer	81,373.56
Finance & Accounting Specialist	70,672.11
Account Manager	63,136.36
Marketing Executive	62,268.52
HR Executive	59,912.98
Operations Executive	57,040.52
IT Support Specialist	55,356.44
Sales Executive	53,438.31
Business Development Executive	50,601.45

Technical designations (Engineering Manager, DevOps Engineer, Software Engineer and Data Analyst) have the highest average salaries. Executive level designations in sales, business development, operations and IT support have the lowest.

Part 4 - Task 4: Salary Levels across Experience Groups and Education Levels

4.1 Average salary by experience

First, the average salary was calculated for every experience value. This gives 231 rows, so only a part of the list is visible. Salary goes up from about 54,900 at 0 years, but the values jump up and down.

```
[40]: experience= df.groupby("Experience")["Salary"].mean() # Calculate average salary by experience
      print(experience)

Experience
0.0      54944.444444
0.1      53021.739130
0.2      56810.344828
0.3      62200.000000
0.4      62414.285714
            ...
24.3     81500.000000
24.4     77666.666667
24.5     83500.000000
24.6    114166.666667
24.9     96750.000000
Name: Salary, Length: 231, dtype: float64
```

Screenshot: average salary for each experience value

4.2 Average salary by experience group

To make the comparison easier, employees were put into three groups: **Low Experience** (0 to 2 years), **Mid Experience** (2 to 10 years) and **High Experience** (more than 10 years). An employee with exactly 2 years is placed in Mid Experience, because the Mid rule is applied after the Low rule.

```
[41]: df["Experience Group"]="High Experience"
      df.loc[df["Experience"].between(0,2),"Experience Group"] = "Low Experience"  # Create experience groups
      df.loc[df["Experience"].between(2,10),"Experience Group"] = "Mid Experience"

[42]: df.groupby("Experience Group")["Salary"].mean()  # Calculate average salary for each experience group

[42]: Experience Group
      High Experience    76881.139489
      Low Experience     60842.135135
      Mid Experience     68445.794421
      Name: Salary, dtype: float64
```

Screenshot: experience groups and their average salary

Experience group	Average salary	Difference from Low group
Low Experience (0 to 2 years)	60,842.14	-
Mid Experience (2 to 10 years)	68,445.79	+7,603.66
High Experience (more than 10 years)	76,881.14	+16,039.00

Salary increases with experience: the High Experience group earns about 26.4% more than the Low Experience group.

4.3 Average salary by education level

```
[43]: experience= df.groupby("Education Level")["Salary"].mean() # Calculate average salary by education level
      print(experience)

Education Level
Bachelor's     65677.423676
Diploma        66905.590106
High School    65412.623377
MBA            72594.660194
Master's       72338.915937
PhD            75347.826087
Name: Salary, dtype: float64
```

Screenshot: average salary by education level

Education level	Average salary
PhD	75,347.83
MBA	72,594.66
Master's	72,338.92
Diploma	66,905.59
Bachelor's	65,677.42
High School	65,412.62

PhD, MBA and Master's holders earn more (72,000 to 75,000) than the other groups (about 65,000 to 67,000). The difference between the highest (PhD) and the lowest (High School) is about 9,935. High School, Bachelor's and Diploma are very close, and Diploma is slightly higher than Bachelor's.

Part 4 - Task 5: Unusually High or Low Salaries (IQR Method)

The interquartile range (IQR) method was used. Q1 is the lower quartile (25%) and Q3 the upper quartile (75%). IQR = Q3 - Q1. A salary below Q1 - 1.5 x IQR or above Q3 + 1.5 x IQR is treated as unusually low or high. The method does not depend on the mean, so extreme salaries do not disturb it.

[51]:
```
Q1 = df["Salary"].quantile(0.25)  # First quartile
Q3 = df["Salary"].quantile(0.75)  # Third quartile

IQR = Q3 - Q1  # Interquartile range

lower_limit = Q1 - 1.5 * IQR  # Lower limit
upper_limit = Q3 + 1.5 * IQR  # Upper limit

outliers = df[
    (df["Salary"] < lower_limit) |  # Below lower limit
    (df["Salary"] > upper_limit)    # Above upper limit
]

print(outliers["Salary"])  # Display outliers
```

Screenshot: code for the IQR method

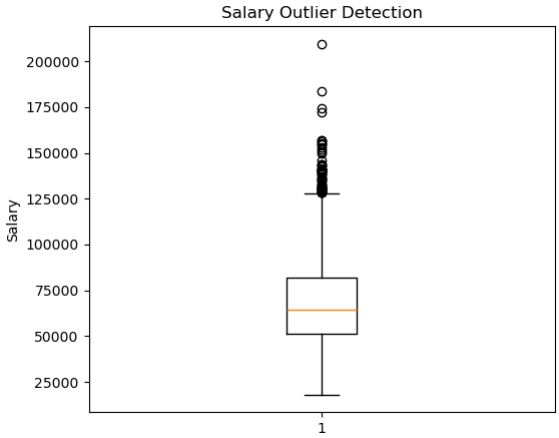
Value	Calculation	Result
Q1 (lower quartile)	25th percentile	51,000
Q3 (upper quartile)	75th percentile	81,875
IQR	81,875 - 51,000	30,875
Lower limit	51,000 - 1.5 x 30,875	4,687.50
Upper limit	81,875 + 1.5 x 30,875	128,187.50

Result: no salary is below the lower limit, but **59 salaries** (2.15% of 2,750 employees) are above the upper limit. They go from 128,500 to 209,544, and the highest value (209,544) is far from the rest. The printed list of the 59 salaries is long, so the code is shown above and the numbers are in the table.

A box plot was drawn to see the same outliers. The box shows the middle 50% of salaries, the orange line is the median and every dot above the top whisker is an outlier.

[53]:
```
# Create a box plot to check salary outliers
plt.figure(figsize=(6,5))
salary = df["Salary"]
plt.boxplot(salary)
plt.title("Salary Outlier Detection")
plt.ylabel("Salary")
plt.show()
```

Screenshot: code for the salary box plot



Screenshot: box plot of salary outliers

What the chart shows: the box lies between 51,000 and 81,875, and all dots are above the top whisker, which matches the IQR result. There are no dots below the box.

Part 4 - Visualization Tasks

Task 6: Box Plots - Salary Distribution across Departments

One box plot was drawn for every department, so that the median, spread and outliers of salaries can be compared.

[60]:

```
departments = df["Department"].unique()
salary = [df[df["Department"] == dept]["Salary"].dropna() for dept in departments]

plt.figure(figsize=(10, 5), facecolor="#F3F6F9")
ax = plt.gca()
ax.set_facecolor("#FFFFFF")

bp = plt.boxplot(salary, patch_artist=True, labels=departments)

for box in bp["boxes"]:
    box.set(facecolor="#8E7DBE", edgecolor="#5B4B8A", linewidth=1.2)

for median in bp["medians"]:
    median.set(color="#D97757", linewidth=2)

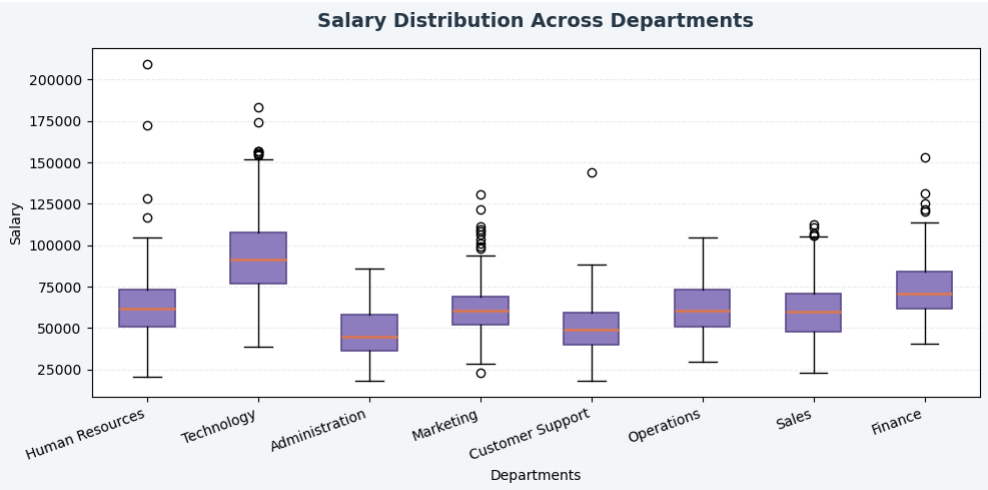
plt.title("Salary Distribution Across Departments",
          fontsize=14, fontweight="bold", color="#263746", pad=15)

plt.xlabel("Departments")
plt.ylabel("Salary")
plt.xticks(rotation=20, ha="right")

plt.grid(axis="y", linestyle="--", color="#D9E1E8", alpha=0.6)
ax.set_axisbelow(True)

plt.tight_layout()
plt.show()
```

Screenshot: code for the department box plots



Screenshot: box plots of salary distribution across departments

What the chart shows: Technology has the highest median (about 91,000) and the whole box is higher than the other departments. Finance is second (median about 71,000). Administration (about 45,000) and Customer Support (about 49,000) have the lowest medians. Human Resources has the widest range of outliers, up to the highest salary of 209,544. Technology, Marketing, Finance and Sales

have several outliers, while Operations and Administration have none. The values are read from the chart, so they are approximate.

Task 7: Scatter Plot - Experience against Salary

Every dot is one employee. The scatter plot shows whether salary goes up with experience.

[55]:

```
experience = df["Experience"]
salary = df["Salary"]

plt.figure(figsize=(8, 5), facecolor="#F3F6F9")
ax = plt.gca()
ax.set_facecolor("#FFFFFF")

plt.scatter(experience, salary, s=65, color="#527DAA",
            edgecolor="#FFFFFF", linewidth=0.8, alpha=0.8)

plt.title("Experience vs Salary",
          fontsize=14, fontweight="bold", color="#263746", pad=15)

plt.xlabel("Experience (Years)")
plt.ylabel("Salary")

plt.grid(linestyle="--", color="#D9E1E8", alpha=0.7)
ax.set_axisbelow(True)

plt.tight_layout()
plt.show()
```

Screenshot: code for the scatter plot



Screenshot: scatter plot of experience against salary

What the chart shows: there is a slight upward movement, because the dots at 10 to 25 years are a little higher than the dots at 0 to 2 years, but the cloud is wide. At every level of experience salaries range from about 20,000 to 125,000, so experience alone does not decide the salary. The highest salary (209,544) belongs to an employee with only about 2 years of experience. There are fewer dots after 15 years because fewer employees have such a long career.

Task 8: Grouped Bar Charts - Average Salary by Designation and Location

The average salary of each designation was calculated for every location. Because there are 15 designations and 14 locations, the designations were divided into three charts of five designations each, ordered by their average salary: the top 5, the next 5 and the last 5.

8.1 Top 5 designations

[59]:

```
data = df.groupby(["Designation", "Location"])["Salary"].mean().unstack()

# Select the 5 designations with the highest average salaries
top5 = df.groupby("Designation")["Salary"].mean().nlargest(5).index

# Professional color palette
colors = ["#365F91", "#5B84B1", "#78A6A6", "#A3B8C8", "#D0A66B"]

ax = data.loc[top5].plot(
    kind="bar",
    figsize=(15, 7),
    width=0.75,
    color=colors[:len(data.columns)],
    edgecolor="white",
    linewidth=0.8)

ax.set_facecolor("#FFFFFF")
ax.figure.set_facecolor("#F3F6F9")

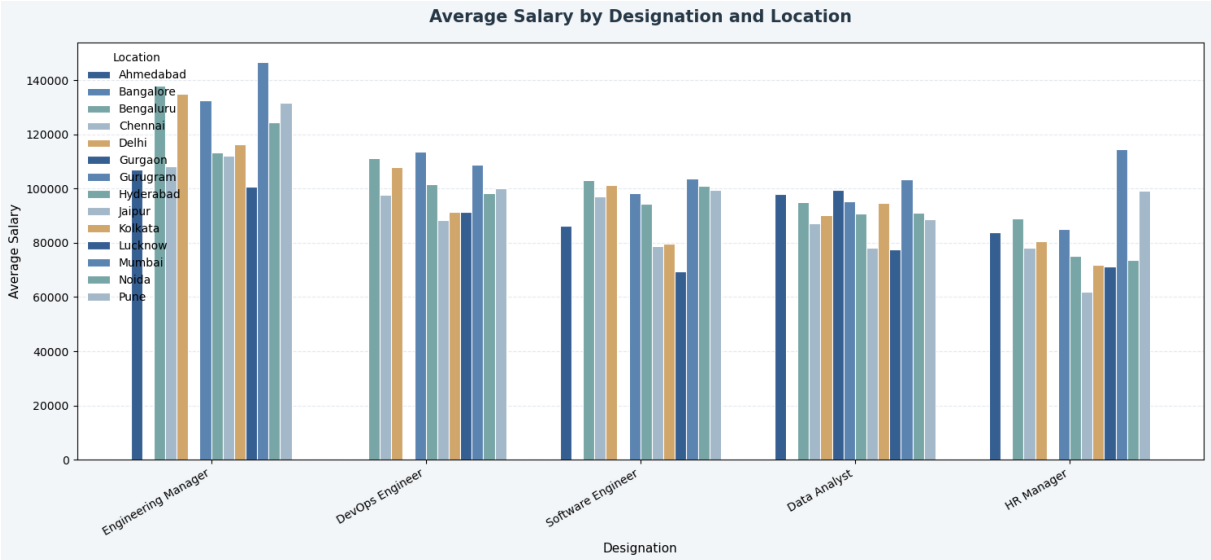
plt.title("Average Salary by Designation and Location",
          fontsize=15, fontweight="bold", color="#263746", pad=18)

plt.xlabel("Designation", fontsize=11)
plt.ylabel("Average Salary", fontsize=11)
plt.xticks(rotation=30, ha="right")

plt.grid(axis="y", linestyle="--", color="#D9E1E8", alpha=0.7)
ax.set_axisbelow(True)

plt.legend(title="Location", frameon=False)
plt.tight_layout()
plt.show()
```

Screenshot: code for the grouped bar chart of the top 5 designations



Screenshot: grouped bar chart of average salary by designation and location (top 5 designations)

What the chart shows: Engineering Manager, DevOps Engineer, Software Engineer, Data Analyst and HR Manager are the five highest paying designations. Engineering Manager has the highest bars in most locations (roughly 100,000 to 146,000), and the average salary of the same designation

changes from location to location. Some locations have no bar for a designation, which means no employee of that designation works there.

8.2 Next 5 designations

[57]:

```
# Calculate average salary by designation and location
data = df.groupby(["Designation", "Location"])["Salary"].mean().unstack()

# Select the next 5 designations after the top 5
next5 = df.groupby("Designation")["Salary"].mean().nlargest(10).index[5:10]

# Professional color palette
colors = ["#365F91", "#5B84B1", "#78A6A6", "#A3B8C8", "#D0A66B"]

# Plot the grouped bar chart
ax = data.loc[next5].plot(
    kind="bar",
    figsize=(15, 7),
    width=0.75,
    color=colors[:len(data.columns)],
    edgecolor="white",
    linewidth=0.8
)

ax.set_facecolor("#FFFFFF")
ax.figure.set_facecolor("#F3F6F9")

plt.title("Average Salary by Designation and Location",
          fontsize=15, fontweight="bold", color="#263746", pad=18)

plt.xlabel("Designation")
plt.ylabel("Average Salary")
plt.xticks(rotation=30, ha="right")

plt.grid(axis="y", linestyle="--", color="#D9E1E8", alpha=0.7)
ax.set_axisbelow(True)

plt.legend(title="Location", frameon=False)
plt.tight_layout()
plt.show()
```

Screenshot: code for the grouped bar chart of the next 5 designations



Screenshot: grouped bar chart of average salary by designation and location (next 5 designations)

What the chart shows: Sales Manager, QA Engineer, Finance & Accounting Specialist, Account Manager and Marketing Executive have lower salaries than the first group, and the difference between locations is again visible.

8.3 Last 5 designations

[58]:

```
# Calculate average salary by designation and location
data = df.groupby(["Designation", "Location"])["Salary"].mean().unstack()

# Select the next 5 designations after the top 10
next5 = df.groupby("Designation")["Salary"].mean().nlargest(15).index[10:15]

# Professional color palette
colors = ["#365F91", "#5B84B1", "#78A6A6", "#A3B8C8", "#D0A66B"]

# Plot the grouped bar chart
ax = data.loc[next5].plot(
    kind="bar",
    figsize=(15, 7),
    width=0.75,
    color=colors[:len(data.columns)],
    edgecolor="white",
    linewidth=0.8
)
ax.set_facecolor("#FFFFFF")
ax.figure.set_facecolor("#F3F6F9")

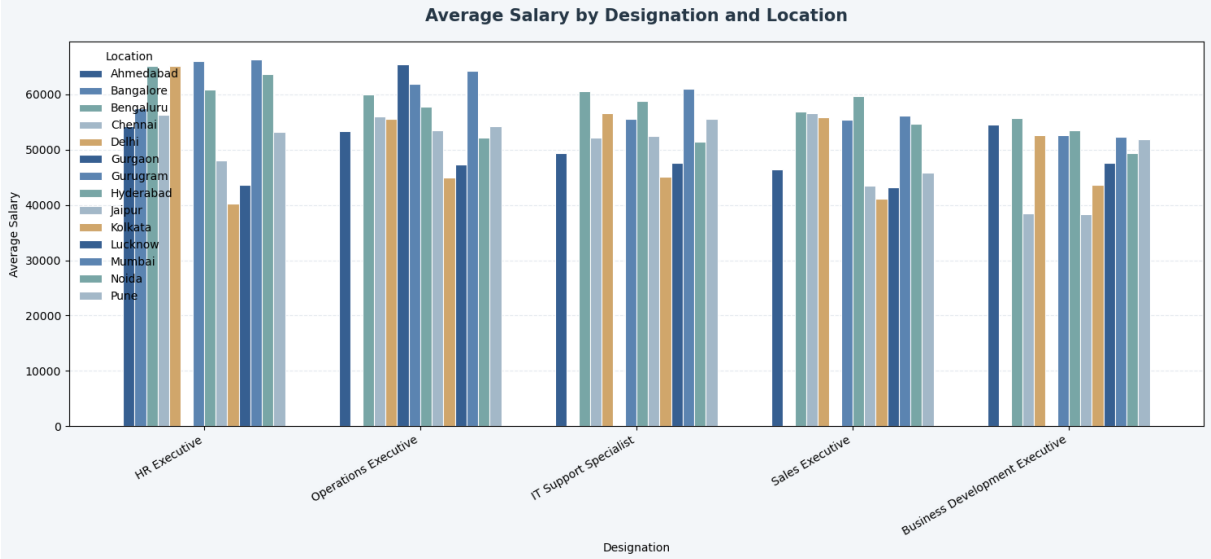
plt.title("Average Salary by Designation and Location",
          fontsize=15, fontweight="bold", color="#263746", pad=18)

plt.xlabel("Designation")
plt.ylabel("Average Salary")
plt.xticks(rotation=30, ha="right")

plt.grid(axis="y", linestyle="--", color="#D9E1E8", alpha=0.7)
ax.set_axisbelow(True)

plt.legend(title="Location", frameon=False)
plt.tight_layout()
plt.show()
```

Screenshot: code for the grouped bar chart of the last 5 designations



Screenshot: grouped bar chart of average salary by designation and location (last 5 designations)

What the chart shows: HR Executive, Operations Executive, IT Support Specialist, Sales Executive and Business Development Executive have the lowest average salaries.

Part 4 - Interpretation

Findings from the analysis

- **Department:** Technology pays the most (93,735.88) and Administration the least (46,757.56). Customer Support is also low (49,741.04).
- **Designation:** Engineering Manager has the highest average salary (127,145.57) and Business Development Executive the lowest (50,601.45). Technical designations earn more than executive level designations.
- **Experience:** salary increases with experience, from 60,842.14 (0 to 2 years) to 76,881.14 (more than 10 years), which is about 26.4% more. The scatter plot shows that experience explains only a part of the pay.
- **Education:** employees with a PhD, MBA or Master's degree earn about 72,000 to 75,000 and others about 65,000 to 67,000. The effect of education is small compared with the effect of department and designation.
- **Unusual salaries:** 59 salaries (2.15%) are above 128,187.50 and none is unusually low. Technology, Finance and Marketing roles, and some Human Resources roles, contain these high salaries.
- **Location:** the average salary of the same designation differs from one location to another.

Limitations

- Missing Salary (22) and Experience (33) values were filled with the median, which may slightly change the true picture.
- The salary and experience averages are descriptive. They show links but do not prove that experience or education causes higher pay.
- The experience groups depend on the limits chosen (2 and 10 years); different limits may give slightly different averages.
- The 59 high salaries were not removed or changed, so the mean (68,370) is slightly pulled up compared with the median (64,500).
- Location names were not cleaned. The grouped bar charts show both Bangalore and Bengaluru and both Gurgaon and Gurugram as separate locations, although they are the same cities, and each chart reuses five colours for 14 locations.
- Performance Rating was not used in this part of the analysis.

Part 4 Conclusion

In Part 4 the salary data was checked and cleaned: 11 department spellings were standardised into 8 departments and 55 missing values (22 in Salary and 33 in Experience) were filled with the median. The average salary of the 2,750 employees is **68,370** (median 64,500, standard deviation 23,979).

Technology is the highest paying department and Engineering Manager the highest paying designation, while Administration and Customer Support pay the least. Salary rises with experience (about 26.4% higher for employees with more than 10 years than for those with up to 2 years) and with higher education, but the effect of education is small. The IQR method found 59 unusually high salaries and none unusually low.

For the company, the pay gaps between departments and between locations for the same designation should be reviewed to keep salaries fair. In future, location names can be standardised, and the link of salary with performance rating can be studied.

PART 5: Employee Training and Skill Development Analysis

About Part 5

Training programs help employees to learn new skills and work better. Part 5 uses an employee training file to check and clean the training scores and costs, calculate how much the scores improve after training, compare training programs, calculate training cost, and see how the training budget is spread across departments.

Part 5 dataset: Task 5.csv - 2,750 rows and 8 columns.

Column	Meaning
Employee ID	Unique identification of each employee
Department	Department in which the employee works
Training Program	Training program attended by the employee
Training Hours	Number of hours spent in training
Pre-Training Score	Score of the employee before the training
Post-Training Score	Score of the employee after the training
Training Cost	Cost of the training for the employee
Productivity Score	Productivity score of the employee

Objectives of Part 5

- To clean the dataset and validate training scores and cost values.
- To calculate the score improvement of each employee and the average improvement of each training program.
- To check the productivity of employees by training program.
- To calculate the training cost per employee and the cost per unit of score improvement.
- To visualise score improvement, training hours and training expenditure with charts.
- To recommend ways to make training more effective.

Quick summary of results (Part 5)

Item	Result
Records in raw file	2,750 rows x 8 columns (22,000 values)
Missing values found	108 (Training Hours 28, Pre-Training Score 18, Post-Training Score 18, Training Cost 24, Productivity Score 20), about 0.49% of all values
Invalid values found	Scores below 0 or above 100 (pre-training -4 and 108, post-training -2 and 115) and one negative Training Cost (-250)
Label problems	Department: 13 different spellings cleaned into 7; Training Program: 14 spellings cleaned into 8
Duplicate employees	None (2,750 unique Employee IDs in 2,750 rows)
How missing values were filled	With the median of each column (hours 36, pre-score 61, post-score 70, cost 478.59, productivity 74.4)
Highest / lowest average score improvement	Data Analytics (11.87) / Compliance Training (6.06)
Total training cost	1,513,976.81 (about 550.54 per employee)
Cost per unit of score improvement	58.69 (about 25,794 improvement points in total, 9.38 per employee)

Item	Result
Highest / lowest department spending	IT (287,695) / Human Resources (146,684)

Part 5 - Task 1: Load the Dataset, Check Its Quality and Clean Training Scores and Cost

First, the libraries were imported and the CSV file was loaded into a Pandas DataFrame called df. The data was then examined for its size, data types, missing values and inconsistent labels, and the problems were fixed before the analysis.

1.1 Import libraries and load the data

Pandas is used for tables and calculations, NumPy for numbers, and Matplotlib for charts. df.sample(5) shows 5 random rows.

```
[1]:
#import libraries :
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

[2]:
# Load the dataset.
df = pd.read_csv(r"C:\Users\vssag\Desktop\Task 5.csv")

[3]:
df.sample(5) # It will give random 5 rows.
```

[3]:

	Employee ID	Department	Training Program	Training Hours	\
1233	EMP01234	Marketing	Data Analytics	84.0	
1294	EMP01295	Customer Support	Data Analytics	74.0	
749	EMP00750	Operations	Compliance Training	8.0	
102	EMP00103	Marketing	Digital Skills	12.0	
1558	EMP01559	IT	Compliance Training	27.0	

	Pre-Training Score	Post-Training Score	Training Cost	\
1233	62.0	73.0	1503.81	
1294	62.0	78.0	978.80	
749	64.0	71.0	100.00	
102	49.0	63.0	267.82	
1558	68.0	74.0	172.66	

	Productivity Score
1233	80.5
1294	89.3
749	89.3
102	90.2
1558	79.7

Screenshot: importing libraries, loading the CSV file and a random sample of 5 rows

1.2 First look at the data

head() shows the first five rows and tail() shows the last five rows. Each row is one employee with the training program, hours, scores, cost and productivity.

```
[4]: df.head() # It shows top five rows

[4]:   Employee ID        Department        Training Program  Training Hours  \
0    EMP00001        Operations  Leadership Development            48.0
1    EMP00002                IT          Data Analytics            75.0
2    EMP00003                IT          Data Analytics            77.0
3    EMP00004  Customer Support      Project Management            16.0
4    EMP00005        Operations          Digital Skills            32.0

   Pre-Training Score  Post-Training Score  Training Cost  Productivity Score
0                73.0                 73.0        1127.71                64.8
1                72.0                 88.0        1336.91                83.5
2                72.0                 84.0        1384.46                88.5
3                56.0                 68.0         234.28                81.0
4                50.0                 63.0         327.50                68.5

[5]: df.tail() # It shows last five rows

[5]:      Employee ID  Department      Training Program  Training Hours  \
2745    EMP02746     Finance    Project Management            57.0
2746    EMP02747          HR  Communication Skills            44.0
2747    EMP02748       Sales   Compliance Training            30.0
2748    EMP02749  Operations      Technical Skills            17.0
2749    EMP02750   Marketing  Communication Skills            35.0

      Pre-Training Score  Post-Training Score  Training Cost  \
2745                82.0                 87.0         602.02
2746                43.0                 48.0         524.49
2747                65.0                 72.0         259.45
2748                77.0                 97.0         179.51
2749                70.0                 84.0         441.53

      Productivity Score
2745                80.8
2746                56.2
2747                82.4
2748                88.0
2749                86.8
```

Screenshot: df.head() and df.tail()

1.3 Shape, column names, data types and summary

The file has 2,750 rows and 8 columns. Training Hours, Pre-Training Score, Post-Training Score, Training Cost and Productivity Score are numbers (float64); Employee ID, Department and Training Program are text. The summary already shows impossible values: the Pre-Training Score has a minimum of -4 and a maximum of 108, the Post-Training Score has a minimum of -2 and a maximum of 115, and the Training Cost has a minimum of -250. A score should be between 0 and 100 and a cost cannot be negative.

```
[6]: df.shape # It will give dataset size(rows,column)

[6]: (2750, 8)

[7]: print(df.columns.tolist()) # It shows All column names in a list.

[7]: ['Employee ID', 'Department', 'Training Program', 'Training Hours', 'Pre-Training Score',
      'Post-Training Score', 'Training Cost', 'Productivity Score']

[8]: df.info() # It will give Dataset structure.

[8]: <class 'pandas.core.frame.DataFrame'>
     RangeIndex: 2750 entries, 0 to 2749
     Data columns (total 8 columns):
      #   Column               Non-Null Count  Dtype
     ---  ------               --------------  -----
      0   Employee ID          2750 non-null   object
      1   Department           2750 non-null   object
      2   Training Program     2750 non-null   object
      3   Training Hours       2722 non-null   float64
      4   Pre-Training Score   2732 non-null   float64
      5   Post-Training Score  2732 non-null   float64
      6   Training Cost        2726 non-null   float64
      7   Productivity Score   2730 non-null   float64
     dtypes: float64(5), object(3)
     memory usage: 172.0+ KB

[9]: df.describe() # It shows statistical summary of the dataset

[9]:        Training Hours  Pre-Training Score  Post-Training Score  Training Cost  \
     count     2722.000000         2732.000000          2732.000000    2726.000000
     mean        37.990816           60.622255            70.001098     550.903177
     std         19.392698           13.036638            13.713646     334.053348
     min          4.000000           -4.000000            -2.000000    -250.000000
     25%         22.000000           52.000000            61.000000     289.037500
     50%         36.000000           61.000000            70.000000     478.565000
     75%         52.000000           69.000000            79.000000     743.895000
     max         90.000000          108.000000           115.000000    1920.670000

            Productivity Score
     count         2730.000000
     mean            74.541978
     std             10.370281
     min             42.400000
     25%             67.700000
     50%             74.400000
     75%             81.500000
     max            100.000000
```

Screenshot: shape, column names, df.info() and df.describe()

1.4 Missing values and unique values

Five columns have missing values: Training Hours (28), Pre-Training Score (18), Post-Training Score (18), Training Cost (24) and Productivity Score (20), 108 in total. Employee ID has 2,750 unique values, so there are no duplicate employees. Department has 13 unique values and Training Program has 14, which is too many and shows spelling problems.

```
[11]: df.isnull().any() # Check any missing value in dataset by each column.

[11]: Employee ID           False
      Department            False
      Training Program      False
      Training Hours         True
      Pre-Training Score     True
      Post-Training Score    True
      Training Cost          True
      Productivity Score     True
      dtype: bool

[12]: df.isnull().sum() # It shows all misiing not/nll in each column.

[12]: Employee ID            0
      Department             0
      Training Program       0
      Training Hours        28
      Pre-Training Score    18
      Post-Training Score   18
      Training Cost         24
      Productivity Score    20
      dtype: int64

[13]: df.nunique() # It will give The count of nunique values.

[13]: Employee ID           2750
      Department              13
      Training Program        14
      Training Hours          87
      Pre-Training Score      78
      Post-Training Score     76
      Training Cost         2658
      Productivity Score     473
      dtype: int64
```

Screenshot: missing values and unique values of every column

1.5 Data consistency: Department and Training Program

Department had 13 spellings (for example "hr", "I.T.", "it", "customer support" and "Cust. Support") and Training Program had 14 (for example "Technical Skill", "technical skills", "Data analytics", "Data Analytic" and "Cust. Service"). After removing spaces, using title case and replacing the short forms, there are 7 departments (Operations, IT, Customer Support, Finance, Sales, Marketing, Human Resources) and 8 training programs (Leadership Development, Data Analytics, Project Management, Digital Skills, Customer Service, Technical Skills, Communication Skills, Compliance Training).

```
[14]: print(df["Department"].unique().tolist()) # It shows unique value in department column.

[14]: ['Operations', 'IT', 'Customer Support', 'Finance', 'Sales', 'Marketing', 'HR', 'hr', 'I.T.',
       'customer support', 'Human Resources', 'it', 'Cust. Support']

[15]: df["Department"] = df["Department"].str.strip().str.title().replace({
          "hr":"Human Resources",
          "hr":"Human Resources",
          "I.T.": "IT",
          "it": "IT",
          "Hr":"Human Resources",
          "It":"IT",
          "Cust. Support":"Customer Support"
      })

[16]: print(df["Department"].unique().tolist()) # Check agian for confirmation.

[16]: ['Operations', 'IT', 'Customer Support', 'Finance', 'Sales', 'Marketing', 'Human Resources']

[17]: print(df["Training Program"].unique().tolist()) # It shows unique value in Training Program col
          umn in a list.

[17]: ['Leadership Development', 'Data Analytics', 'Project Management', 'Digital Skills', 'Customer
       Service', 'Technical Skills', 'Communication Skills', 'Compliance Training', 'Data
       analytics', 'Technical Skill', 'Cust. Service', 'technical skills', 'Data Analytic',
       'Customer service']

[18]: df["Training Program"] = df["Training Program"].str.strip().str.title().replace({
          "Technical Skill":"Technical Skills",
          "Data Analytic":"Data Analytics",
          "Cust. Service":"Customer Service"

      })

[19]: print(df["Training Program"].unique().tolist())

[19]: ['Leadership Development', 'Data Analytics', 'Project Management', 'Digital Skills', 'Customer
       Service', 'Technical Skills', 'Communication Skills', 'Compliance Training']
```

Screenshot: department and training program names before and after cleaning

1.6 Training Hours

Training Hours are from 4 to 90 with a median of 36, so no value is impossible. The 28 missing values were filled with the median, and the check shows 0 missing.

```
[22]: df["Training Hours"].describe()

[22]: count    2722.000000
      mean       37.990816
      std        19.392698
      min         4.000000
      25%        22.000000
      50%        36.000000
      75%        52.000000
      max        90.000000
      Name: Training Hours, dtype: float64

[23]: df["Training Hours"] = df["Training Hours"].fillna(df["Training Hours"].median()) # Replace wit
          h median.

[24]: df["Training Hours"].isna().sum() # Check again for confiration.

[24]: np.int64(0)
```

Screenshot: checking and filling Training Hours

1.7 Pre-Training Score

Scores below 0 or above 100 are not valid, so they were set to missing (2 values). After this the minimum is 20 and the maximum is 95. The histogram has a bell shape with most scores between about 50 and 75. The missing values were filled with the median, 61.

```
[25]: df["Pre-Training Score"].describe() # Stastical summary

[25]: count    2732.000000
      mean       60.622255
      std        13.036638
      min        -4.000000
      25%        52.000000
      50%        61.000000
      75%        69.000000
      max       108.000000
      Name: Pre-Training Score, dtype: float64

[26]: df.loc[(df["Pre-Training Score"] < 0) | (df["Pre-Training Score"] > 100),
          "Pre-Training Score"] = None # # Handle invalid scores

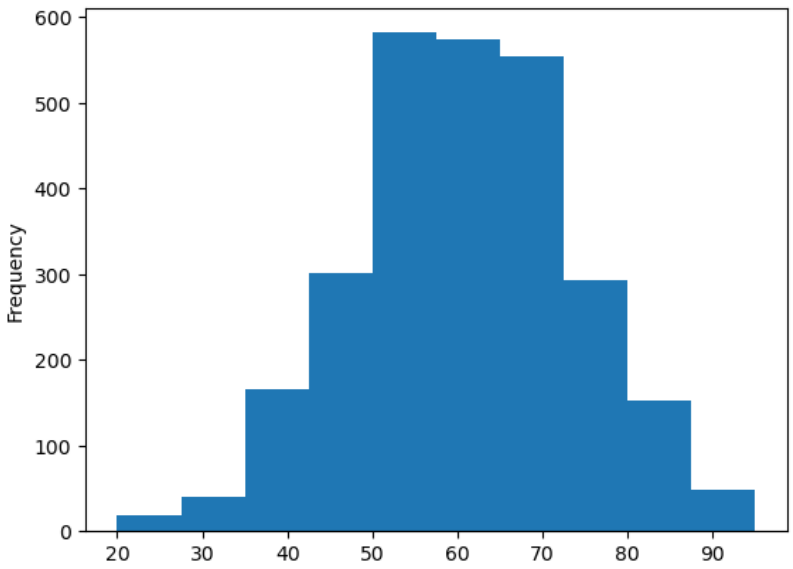
[27]: df["Pre-Training Score"].describe() # # Recheck summary

[27]: count    2730.000000
      mean       60.628571
      std        12.950895
      min        20.000000
      25%        52.000000
      50%        61.000000
      75%        69.000000
      max        95.000000
      Name: Pre-Training Score, dtype: float64

[28]: df["Pre-Training Score"].plot.hist() # # Check score distribution
      plt.show()

[29]: df["Pre-Training Score"] = df["Pre-Training Score"].fillna(df["Pre-Training Score"].median())
                                        # # Fill missing values with the median
```

Screenshot: validating and filling Pre-Training Score



Screenshot: histogram of Pre-Training Score

1.8 Post-Training Score

The same rule was used for the Post-Training Score: values below 0 or above 100 were set to missing and all missing values were filled with the median, 70. After this the minimum is 25, the maximum is

100 and the mean is 70.01. The histogram is shifted to the right compared with the pre-training one, which shows that scores are higher after training.

```
[30]: df["Post-Training Score"].describe()

[30]: count    2732.000000
      mean       70.001098
      std        13.713646
      min        -2.000000
      25%        61.000000
      50%        70.000000
      75%        79.000000
      max       115.000000
      Name: Post-Training Score, dtype: float64

[31]: df.loc[(df["Post-Training Score"] < 0) | (df["Post-Training Score"] > 100),
          "Post-Training Score"] = None # # Handle invalid scores

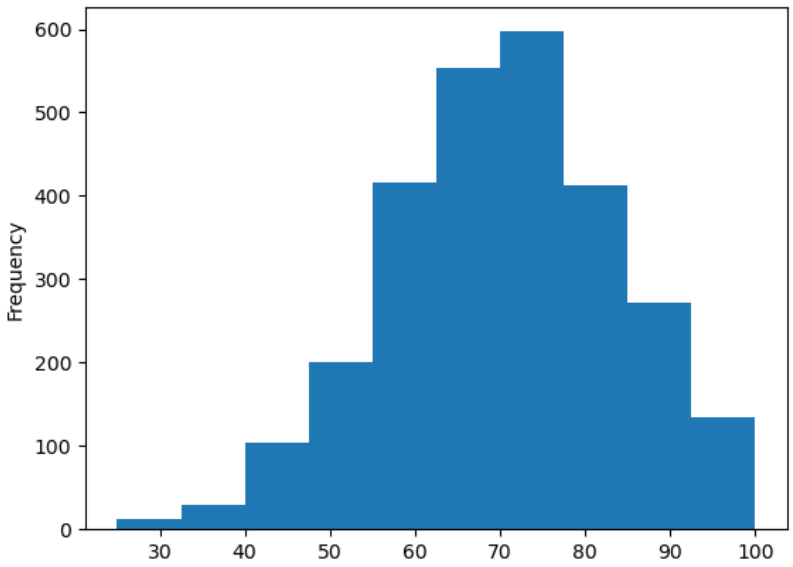
[32]: df["Post-Training Score"].plot.hist()

[33]: df["Post-Training Score"] = df["Post-Training Score"].fillna(df["Post-Training Score"].median()
          )
                                              # Fill missing values with the median

[34]: df["Post-Training Score"].describe() # Recheck summary

[34]: count    2750.000000
      mean       70.010909
      std        13.572404
      min        25.000000
      25%        61.000000
      50%        70.000000
      75%        79.000000
      max       100.000000
      Name: Post-Training Score, dtype: float64
```

Screenshot: validating and filling Post-Training Score



Screenshot: histogram of Post-Training Score

1.9 Training Cost

One cost was negative (-250) and was set to missing. After this the minimum is 100, the maximum is 1,920.67 and the median is 478.59. The histogram is skewed to the right: most trainings cost less than 1,000 and only a few cost more than 1,500. The missing values were filled with the median.

```
[35]: df["Training Cost"].describe()

[35]: count    2726.000000
      mean      550.903177
      std       334.053348
      min      -250.000000
      25%       289.037500
      50%       478.565000
      75%       743.895000
      max      1920.670000
      Name: Training Cost, dtype: float64

[36]: df.loc[df["Training Cost"] < 0, "Training Cost"] = None # Handle invalid scores

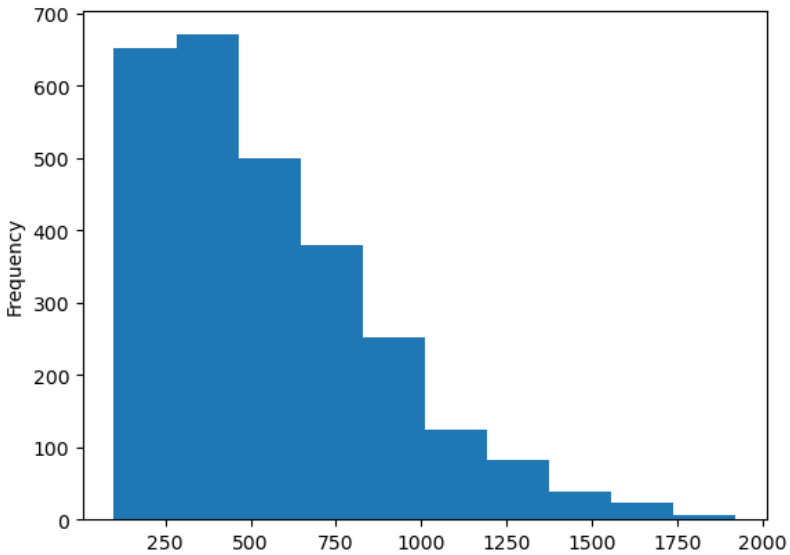
[37]: df["Training Cost"].describe()

[37]: count    2725.000000
      mean      551.197086
      std       333.761951
      min       100.000000
      25%       289.090000
      50%       478.590000
      75%       744.060000
      max      1920.670000
      Name: Training Cost, dtype: float64

[38]: df["Training Cost"].plot.hist()

[39]: df["Training Cost"] = df["Training Cost"].fillna(df["Training Cost"].median()) # Fill missing v
      alues with the median
```

Screenshot: validating and filling Training Cost



Screenshot: histogram of Training Cost

1.10 Productivity Score

Productivity Score is from 42.4 to 100 with a median of 74.4, and all values are valid. The 20 missing values were filled with the median, and the final check shows 0 missing values in every column.

```
[40]: df["Productivity Score"].describe()

[40]: count    2730.000000
      mean       74.541978
      std        10.370281
      min        42.400000
      25%        67.700000
      50%        74.400000
      75%        81.500000
      max       100.000000
      Name: Productivity Score, dtype: float64

[41]: df["Productivity Score"] = df["Productivity Score"].fillna(df["Productivity Score"].median())

[42]: df.isna().sum() # Recheck

[42]: Employee ID            0
      Department             0
      Training Program       0
      Training Hours         0
      Pre-Training Score     0
      Post-Training Score    0
      Training Cost          0
      Productivity Score     0
      dtype: int64
```

Screenshot: checking and filling Productivity Score, and the final check of missing values

Column	Problem found	How it was handled
Department	13 spellings	Cleaned into 7 departments
Training Program	14 spellings	Cleaned into 8 training programs
Training Hours	28 missing	Filled with the median (36)
Pre-Training Score	18 missing; values -4 and 108	Values outside 0 to 100 set to missing; filled with the median (61)
Post-Training Score	18 missing; values -2 and 115	Values outside 0 to 100 set to missing; filled with the median (70)
Training Cost	24 missing; one value of -250	Negative value set to missing; filled with the median (478.59)
Productivity Score	20 missing	Filled with the median (74.4)

Part 5 - Task 2: Calculate the Score Improvement for Each Employee

The improvement of each employee is the Post-Training Score minus the Pre-Training Score. For example, the first employee has an improvement of 0, the second 16 and the third 12.

```
[48]: df["improvement_score"] = df["Post-Training Score"] - df["Pre-Training Score"]
      print(improvement_score)
      #print(improvement_score)

[48]: 0        0.0
      1       16.0
      2       12.0
      3       12.0
      4       13.0
              ...
      2745     5.0
      2746     5.0
      2747     7.0
      2748    20.0
      2749    14.0
      Length: 2750, dtype: float64
```

Screenshot: calculating the score improvement for each employee

- **Improvement can be zero or negative.** The scatter plot in Task 7 shows employees with 0 improvement and a few with a negative improvement (down to about -22), which means that their score was lower after training.
- **Filled values can create artificial improvement.** For the 18 employees whose pre-score and the 18 whose post-score were missing, the median (61 or 70) was used, so their improvement is an estimate.

Part 5 - Task 3: Average Improvement and Average Productivity by Training Program

The data was grouped by Training Program, and the average score improvement and the average Productivity Score were calculated for each program.

```
[49]: df.groupby("Training Program")["improvement_score"].mean()

[49]: Training Program
      Communication Skills       8.019277
      Compliance Training        6.063241
      Customer Service           8.929245
      Data Analytics            11.870206
      Digital Skills            10.588462
      Leadership Development     7.094340
      Project Management         9.919540
      Technical Skills          11.069418
      Name: improvement_score, dtype: float64

[50]: df.groupby("Training Program")["Productivity Score"].mean() # Calculate average productivity sc
          ore by training program

[50]: Training Program
      Communication Skills      73.817831
      Compliance Training       72.610277
      Customer Service          74.063679
      Data Analytics            76.428909
      Digital Skills            74.672692
      Leadership Development    72.940755
      Project Management        74.660536
      Technical Skills          75.872045
      Name: Productivity Score, dtype: float64
```

Screenshot: average score improvement and average productivity score by training program

Training program	Average score improvement	Average productivity score
Data Analytics	11.87	76.43
Technical Skills	11.07	75.87
Digital Skills	10.59	74.67
Project Management	9.92	74.66
Customer Service	8.93	74.06
Communication Skills	8.02	73.82
Leadership Development	7.09	72.94
Compliance Training	6.06	72.61

- **Data Analytics (11.87), Technical Skills (11.07) and Digital Skills (10.59) give the biggest improvement.** Compliance Training (6.06) and Leadership Development (7.09) give the smallest. The best program improves scores almost twice as much as the weakest (11.87 against 6.06).
- **The productivity score follows the same order.** Data Analytics has the highest average productivity (76.43) and Compliance Training the lowest (72.61), but the gap is only 3.8 points, so the link is weak.
- **Technical and digital programs work better than soft-skill and compliance programs** in this data.

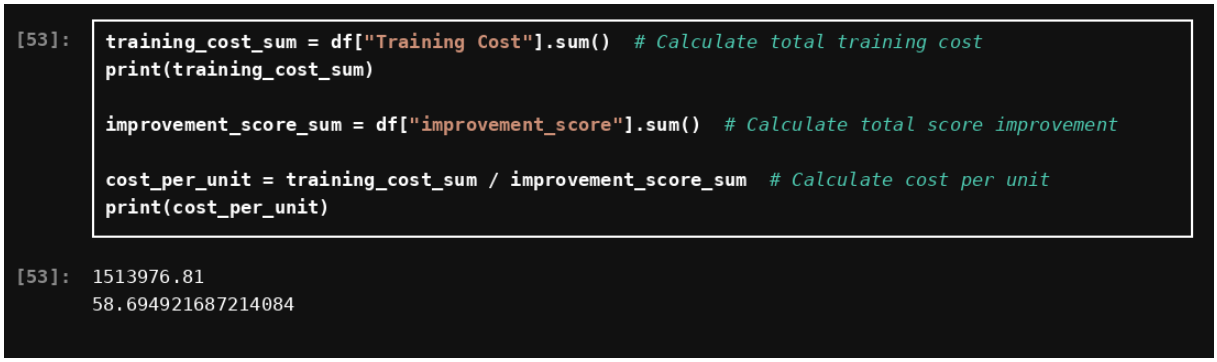
Part 5 - Task 4: Productivity Before and After Training

The task asks to compare productivity before and after training, if both measurements are available. The file has only one Productivity Score for each employee and no separate productivity before and after the training, so this comparison is not possible. What the data does allow is the comparison of the training test scores (Pre-Training and Post-Training Score), which was done in Tasks 2 and 3, and the average productivity of each program shown in Task 3.

To measure the effect of training on productivity, productivity should be recorded both before and after the training.

Part 5 - Task 5: Training Cost per Employee and Cost per Unit of Score Improvement

The total training cost was added up and divided by the total score improvement of all employees.



```
[53]: training_cost_sum = df["Training Cost"].sum()  # Calculate total training cost
      print(training_cost_sum)

      improvement_score_sum = df["improvement_score"].sum()  # Calculate total score improvement

      cost_per_unit = training_cost_sum / improvement_score_sum  # Calculate cost per unit
      print(cost_per_unit)

[53]: 1513976.81
      58.694921687214084
```

Screenshot: total training cost and cost per unit of score improvement

Measure	Value
Total training cost	1,513,976.81
Training cost per employee (total cost / 2,750 employees, calculated from the total)	about 550.54
Cost per unit of score improvement	58.69
Total score improvement (total cost / 58.69, calculated)	about 25,794 points (9.38 per employee)

- **Every point of score improvement costs about 58.69 on average.** The notebook calculates this for all employees together; it does not give the cost per improvement for each program, which would show the program with the best value.
- **The notebook does not print the cost per employee;** it is calculated here from the total cost and the number of employees.

Part 5 - Visualization Tasks

Task 6: Bar Chart - Average Score Improvement across Training Programs

A bar chart compares the average score improvement of the 8 training programs, with the value written above each bar.

[57]:

```
plt.figure(figsize=(12, 6), facecolor="#F3F6F9")

ax = plt.gca()
ax.set_facecolor("#FFFFFF")

programs_name = improvement_score_avg.index
programs_avg = improvement_score_avg.values

colors = ["#365F91", "#527DAA", "#6A91B5", "#78A6A6",
          "#8FAFC4", "#A3B8C8", "#648C9E", "#789B9B"]

bars = plt.bar(
    programs_name, programs_avg,
    color=colors[:len(programs_name)],
    edgecolor="white",
    linewidth=1.2,
    width=0.65
)

plt.title("Average Score Improvement Across Training Programs",
          fontsize=14, fontweight="bold", color="#263746", pad=16)

plt.xlabel("Training Programs", fontsize=11, color="#455A64")
plt.ylabel("Average Score Improvement", fontsize=11, color="#455A64")

plt.xticks(rotation=30, ha="right")
plt.yticks(color="#455A64")

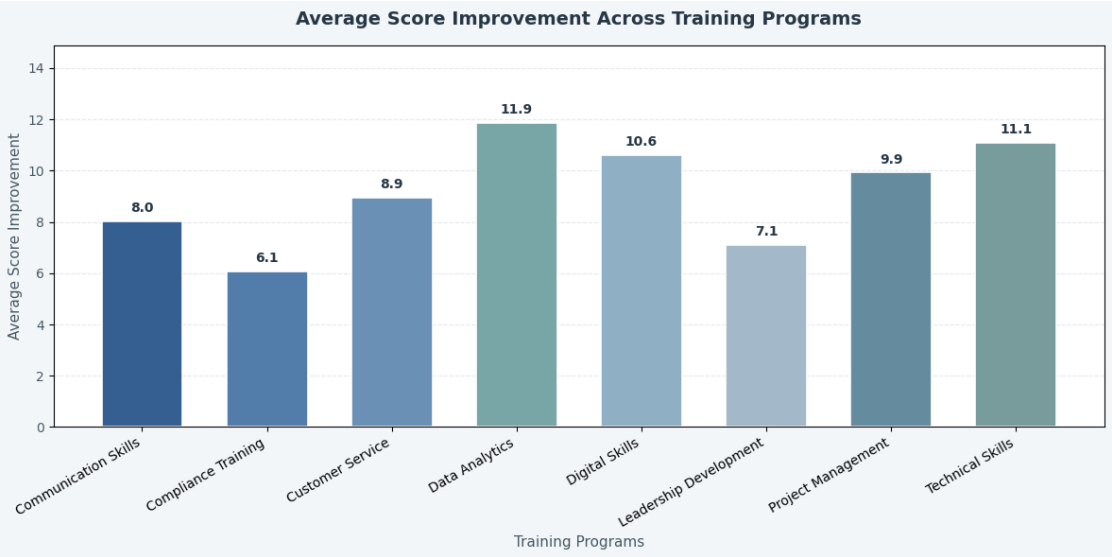
plt.ylim(0, max(programs_avg) + 3)

plt.bar_label(bars, fmt="%.1f", padding=5,
              fontsize=10, fontweight="bold", color="#263746")

plt.grid(axis="y", linestyle="--", color="#D9E1E8", alpha=0.7)
ax.set_axisbelow(True)

plt.tight_layout()
plt.show()
```

Screenshot: code for the bar chart



Screenshot: bar chart of average score improvement across training programs

- **Data Analytics (11.9) is the tallest bar and Compliance Training (6.1) the shortest.** The chart shows two groups: Data Analytics, Technical Skills and Digital Skills (10.6 to 11.9), and the other programs (6.1 to 9.9).

Task 7: Scatter Plot - Training Hours against Score Improvement

Each point is one employee. The horizontal axis is Training Hours and the vertical axis is the Score Improvement.

[59]:

```
hours = df["Training Hours"]  # Get training hours

plt.figure(figsize=(9, 5), facecolor="#F1F5F9")
ax = plt.gca()
ax.set_facecolor("#FFFFFF")

plt.scatter(
    hours, improvement_score,
    s=65,
    color="#3B6F96",
    edgecolor="#1E4057",
    linewidth=0.8,
    alpha=0.75
)

plt.title("Score Improvement vs Training Hours",
          fontsize=14, fontweight="bold", color="#1E293B", pad=16)

plt.xlabel("Training Hours", fontsize=11, color="#475569")
plt.ylabel("Score Improvement", fontsize=11, color="#475569")

plt.grid(linestyle="--", color="#CBD5E1", alpha=0.7)
ax.set_axisbelow(True)

plt.tight_layout()
plt.show()
plt.show()
```

Screenshot: code for the scatter plot



Screenshot: scatter plot of score improvement against training hours

- **There is no clear relationship.** At every number of training hours (from 4 to 90) most improvements are between 0 and 20 points, so more hours do not clearly give more improvement.
- **Many employees have an improvement of exactly 0 (the flat line at the bottom of the points),** and a few have a negative improvement, down to about -22.
- **The biggest improvements (about 30 to 33 points) happen at short and medium training hours (about 18 to 40 hours).** Very long trainings (more than 80 hours) do not give bigger improvements.

Task 8: Bar Chart - Training Expenditure across Departments

The Training Cost was added up for each department and drawn as a bar chart with the total above each bar.

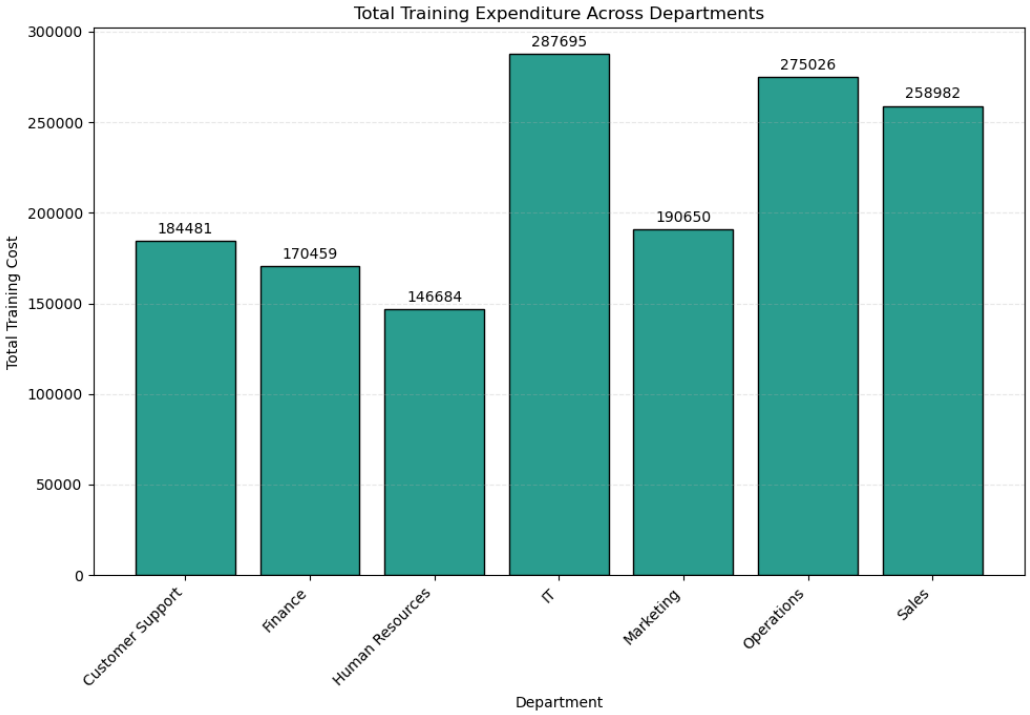
```
training_cost = df.groupby("Department")["Training Cost"].sum()  # Calculate total training cos
    t

departments = training_cost.index
cost = training_cost.values

plt.figure(figsize=(10, 7))
bars = plt.bar(departments, cost, color="#2A9D8F", edgecolor="black", linewidth=1)

plt.title("Total Training Expenditure Across Departments")
plt.xlabel("Department")
plt.ylabel("Total Training Cost")
plt.xticks(rotation=45, ha="right")
plt.grid(axis="y", linestyle="--", alpha=0.3)
plt.bar_label(bars, fmt="%.0f", padding=3)
plt.tight_layout()
plt.show()
```

Screenshot: code for the department expenditure chart



Screenshot: bar chart of total training expenditure across departments

Department	Total training cost	Share of the total
IT	287,695	19.0%
Operations	275,026	18.2%
Sales	258,982	17.1%
Marketing	190,650	12.6%
Customer Support	184,481	12.2%
Finance	170,459	11.3%
Human Resources	146,684	9.7%

- **IT (287,695), Operations (275,026) and Sales (258,982) spend the most,** together about 54% of the total budget. Human Resources (146,684) spends the least.
- **These are totals,** so they depend on the number of employees in each department, which was not calculated. The cost per employee would be a fairer comparison.

Part 5 - Interpretation

Findings from the analysis

- **Scores are higher after training:** the average score rises by about 9.4 points per employee (the median goes from 61 to 70).
- **Programs:** Data Analytics (11.87), Technical Skills (11.07) and Digital Skills (10.59) improve scores the most; Compliance Training (6.06) and Leadership Development (7.09) improve them the least.
- **Training hours:** more training hours do not clearly give more improvement, so training time can be used more efficiently.
- **Cost:** the total training cost is 1,513,976.81, about 550.54 per employee and 58.69 for every point of score improvement.
- **Departments:** IT, Operations and Sales spend about 54% of the budget; Human Resources spends the least (9.7%).

Limitations

- Productivity before and after training is not available, so the effect of training on productivity could not be measured.
- 108 missing values were filled with the median, including 36 pre- or post-training scores, which can create artificial improvements.
- Some employees show a negative improvement and many show exactly 0; these cases were not checked individually.
- The cost per program and the cost per employee in each department were not calculated.
- The results are descriptive; they show links but do not prove that a program causes the improvement.

Recommendations to make training more effective

1. **Invest more in the best programs:** Data Analytics, Technical Skills and Digital Skills give the highest improvement, so they can be offered to more employees.
2. **Redesign the weakest programs:** review the content and the teaching method of Compliance Training (6.06) and Leadership Development (7.09), for example with practical exercises.
3. **Do not rely on more hours:** longer training does not give more improvement, so use shorter, focused sessions with follow-up practice.
4. **Measure cost per improvement for each program:** choose the programs with the lowest cost for each point gained.
5. **Measure productivity before and after training:** record productivity at both times so that the effect of training on work results can be shown.
6. **Review the budget split:** compare the spending per employee in each department and give more support to departments with a low improvement.
7. **Support employees with no or negative improvement:** find out why they did not benefit (wrong level, wrong program, no support) and offer another program.
8. **Improve data quality:** accept only scores from 0 to 100 and costs above 0, use fixed lists for department and program names, and avoid filling missing scores with medians.

Part 5 Conclusion

In Part 5 the training data was checked and cleaned: 13 department spellings and 14 training program spellings were standardised, the impossible scores and the negative cost were set to missing, and 108 missing values were filled with the median. After cleaning, the average score improves by about **9.4 points** after training.

Data Analytics (11.87), Technical Skills (11.07) and Digital Skills (10.59) give the best improvement, while **Compliance Training (6.06) and Leadership Development (7.09)** give the least. Training hours show no clear link with improvement. The total training cost is **1,513,976.81** (about 550.54 per employee and 58.69 per point of improvement), and IT, Operations and Sales spend the most.

For the company, the best programs should be extended and the weak programs redesigned. In future, productivity should be recorded before and after training, and the cost per improvement should be calculated for each program and each department.